

Chapter 4

Tree

Introduction to Tree Data Structure

Tree data structure is a hierarchical structure that is used to represent and organize data in a way that is easy to navigate and search. It is a collection of nodes that are connected by edges and has a hierarchical relationship between the nodes.

The topmost node of the tree is called the **root**, and the nodes below it are called the child nodes. Each node can have multiple child nodes, and these child nodes can also have their own child nodes, forming a recursive structure.

Why Tree is considered a non-linear data structure?

The data in a tree are not stored in a sequential manner i.e., they are not stored linearly. Instead, they are arranged on multiple levels or we can say it is a hierarchical structure. For this reason, the tree is considered to be a non-linear data structure.

Importance for Tree Data Structure:

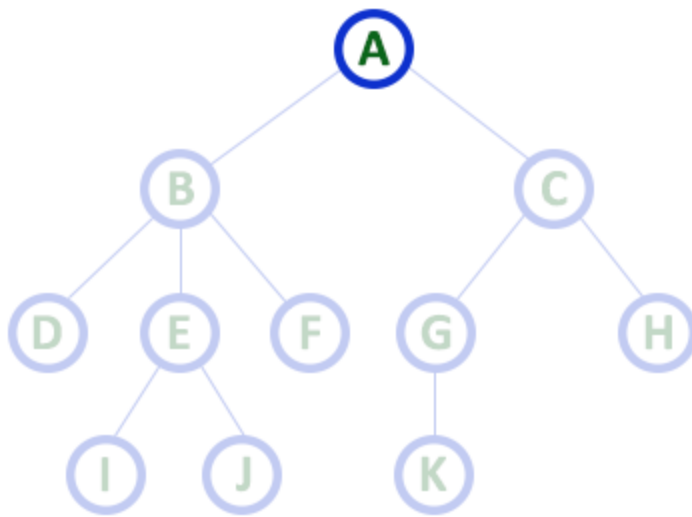
One reason to use trees might be because you want to store information that naturally forms a hierarchy. For example, the file system on a computer: The DOM model of an HTML page is also tree where we have html tag as root, head and body its children and these tags, then have their own children.

Tree Terminology

In a tree data structure, we use the following terminology...

1. Root

In a tree data structure, the first node is called as **Root Node**. Every tree must have a root node. We can say that the root node is the origin of the tree data structure. In any tree, there must be only one root node. We never have multiple root nodes in a tree.

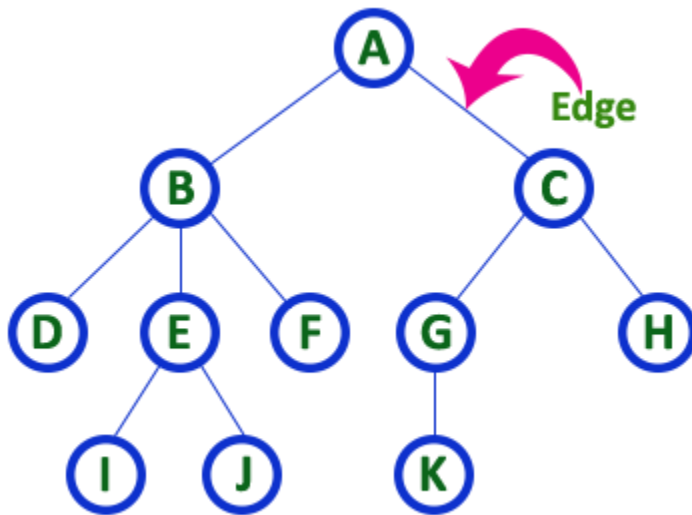


Here 'A' is the 'root' node

- In any tree the first node is called as ROOT node

2. Edge

In a tree data structure, the connecting link between any two nodes is called as **EDGE**. In a tree with 'N' number of nodes there will be a maximum of 'N-1' number of edges.

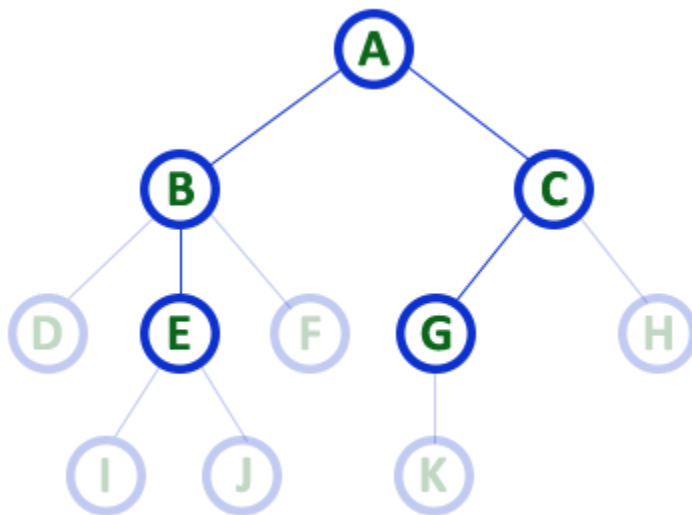


- In any tree, 'Edge' is a connecting link between two nodes.

3. Parent

In a tree data structure, the node which is a predecessor of any node is called as **PARENT NODE**. In simple words, the node which has a branch from it to

any other node is called a parent node. Parent node can also be defined as "The node which has child / children".

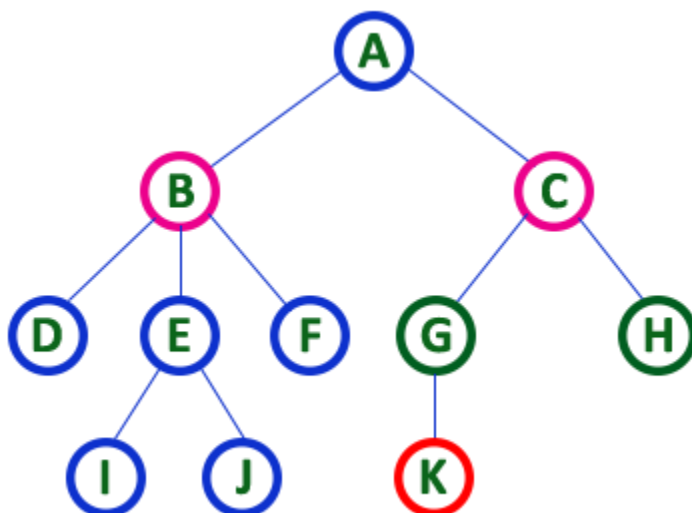


Here A, B, C, E & G are **Parent** nodes

- In any tree the node which has child / children is called 'Parent'
- A node which is predecessor of any other node is called 'Parent'

4. Child

In a tree data structure, the node which is descendant of any node is called as **CHILD Node**. In simple words, the node which has a link from its parent node is called as child node. In a tree, any parent node can have any number of child nodes. In a tree, all the nodes except root are child nodes.



Here B & C are **Children** of A

Here G & H are **Children** of C

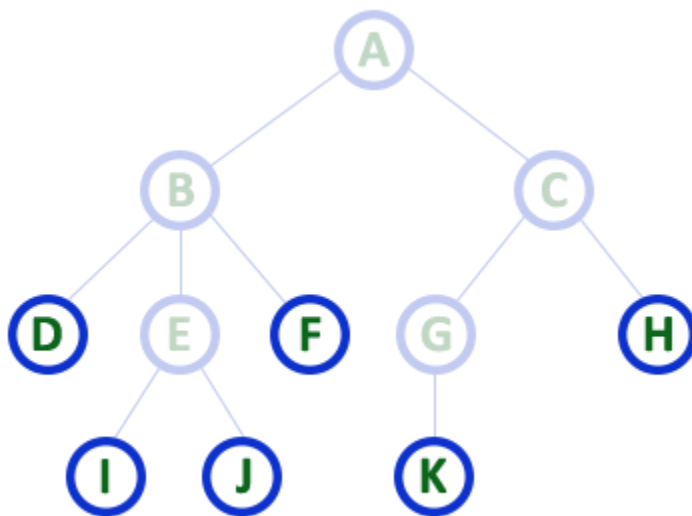
Here K is **Child** of G

- descendant of any node is called as **CHILD Node**

6. Leaf

In a tree data structure, the node which does not have a child is called as **LEAF Node**. In simple words, a leaf is a node with no child.

In a tree data structure, the leaf nodes are also called as **External Nodes**. External node is also a node with no child. In a tree, leaf node is also called as 'Terminal' node.



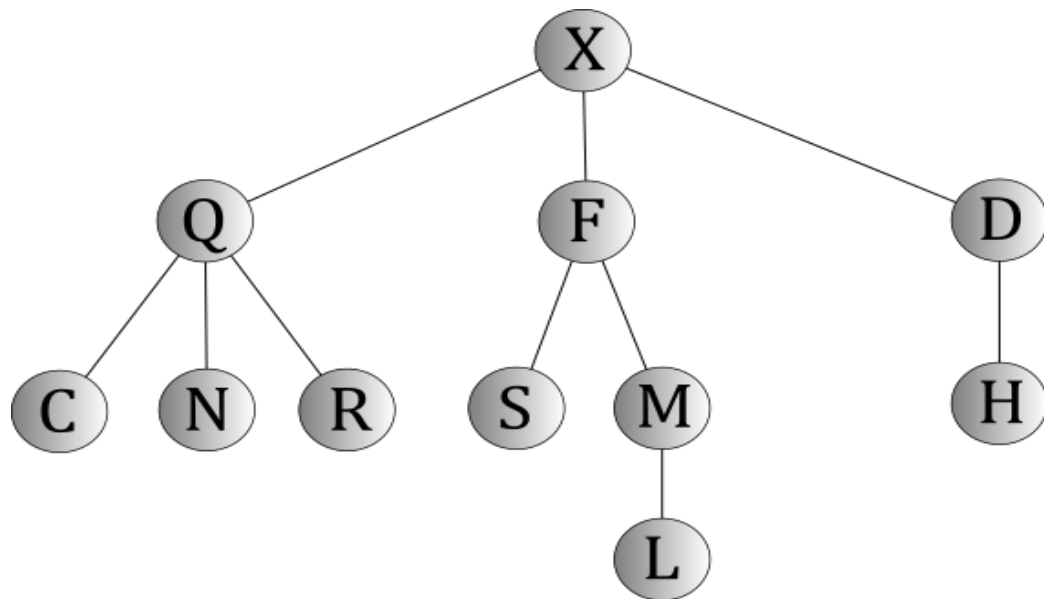
Here D, I, J, F, K & H are **Leaf** nodes

- In any tree the node which does not have children is called '**Leaf**'
- A node without successors is called a '**leaf**' node

Note: If tree have n nodes then it must have $n-1$ edges.

2. General Tree:

In the data structure, **General tree** is a tree in which each node can have either zero or many child nodes. It can not be empty. In general tree, there is no limitation on the degree of a node.



2.1. Application of Genaral Tree:

File systems

Trees are a natural way to organize data in a file system, where each folder has subfolders

Organizational charts

A company's structure can be modeled with a tree, where the president is the root node, vice presidents are at level 1, and subordinates are at lower levels

Web pages

The DOM (Document Object Model) in web browsers is a tree structure, where the HTML tag is the root, and head and body are children

AI and Machine Learning

- **Game Trees:**

- In decision-making problems like chess or tic-tac-toe, general trees represent possible game states and moves.

2.2. Game Tree:

Game Tree: A Visual Representation of Game Strategies

What is a Game Tree?

In game theory and artificial intelligence, a game tree is a graphical representation used to visualize and analyze the possible moves, decisions, and their subsequent outcomes in a sequential game. It's essentially a decision tree tailored for strategic games.

A **game tree** is a tree-like data structure used to represent all possible moves in a game from a given initial state. It is commonly used in decision-making and game theory for two-player games like chess, tic-tac-toe, or checkers.

Key Components of a Game Tree:

1. Nodes:

- Represent game states or decision points.
- Can be classified as:
 - **Root Node:** The starting point of the game.
 - **Decision Nodes:** Points where a player makes a choice.
 - **Chance Nodes:** Points where an uncertain event occurs (e.g., rolling a dice).
 - **Terminal Nodes:** End points of the game, representing final outcomes or payoffs.

2. Edges:

- Connect nodes, representing possible moves or choices.
- Each edge is associated with a probability (in chance nodes) or a decision.

Applications:

- AI for games like chess, tic-tac-toe, and Go.
- Decision-making in competitive scenarios.
- Simulating possible outcomes for planning and analysis.

Example: Tic-Tac-Toe Game Tree

How Game Trees are Used:

- **Decision Making:**
 - Players can use game trees to evaluate different strategies and choose the optimal moves.
 - Algorithms like Minimax and Alpha-Beta pruning can be applied to efficiently search game trees and find the best moves.
- **Game Analysis:**
 - Game theorists use game trees to analyze the structure of games, identify equilibrium points, and predict player behavior.
- **Artificial Intelligence:**
 - Game trees are fundamental in AI for developing game-playing agents.
 - AI algorithms like Monte Carlo Tree Search (MCTS) use game trees to explore possible moves and make decisions in complex games like Go and Chess.

Advantages of Game Trees:

- **Visual Clarity:** Provide a clear and intuitive representation of game dynamics.
- **Comprehensive Analysis:** Allow for the exploration of all possible scenarios.
- **Optimal Decision Making:** Help players make informed choices and improve their strategies.

Limitations of Game Trees:

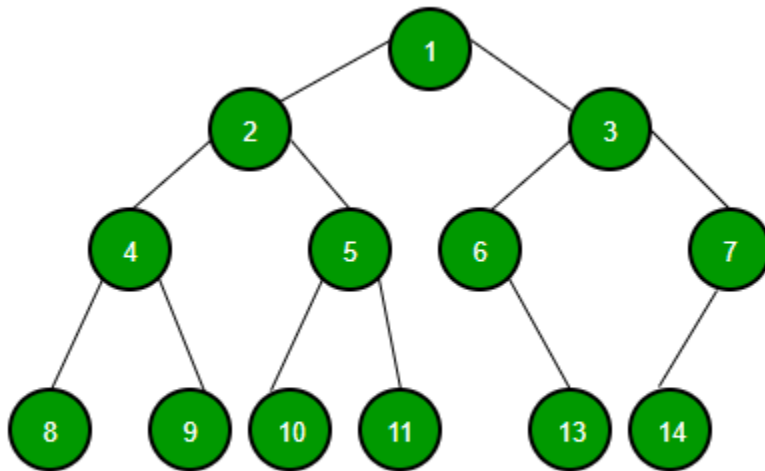
- **Complexity:** For complex games with many possible moves, game trees can become extremely large and computationally expensive to analyze.
- **Imperfect Information:** Games with hidden information (e.g., card games) can be difficult to model accurately using game trees.

Game trees help in analyzing strategies, predicting opponent moves, and building AI that can play games effectively.

3. Binary Tree Data Structure

-

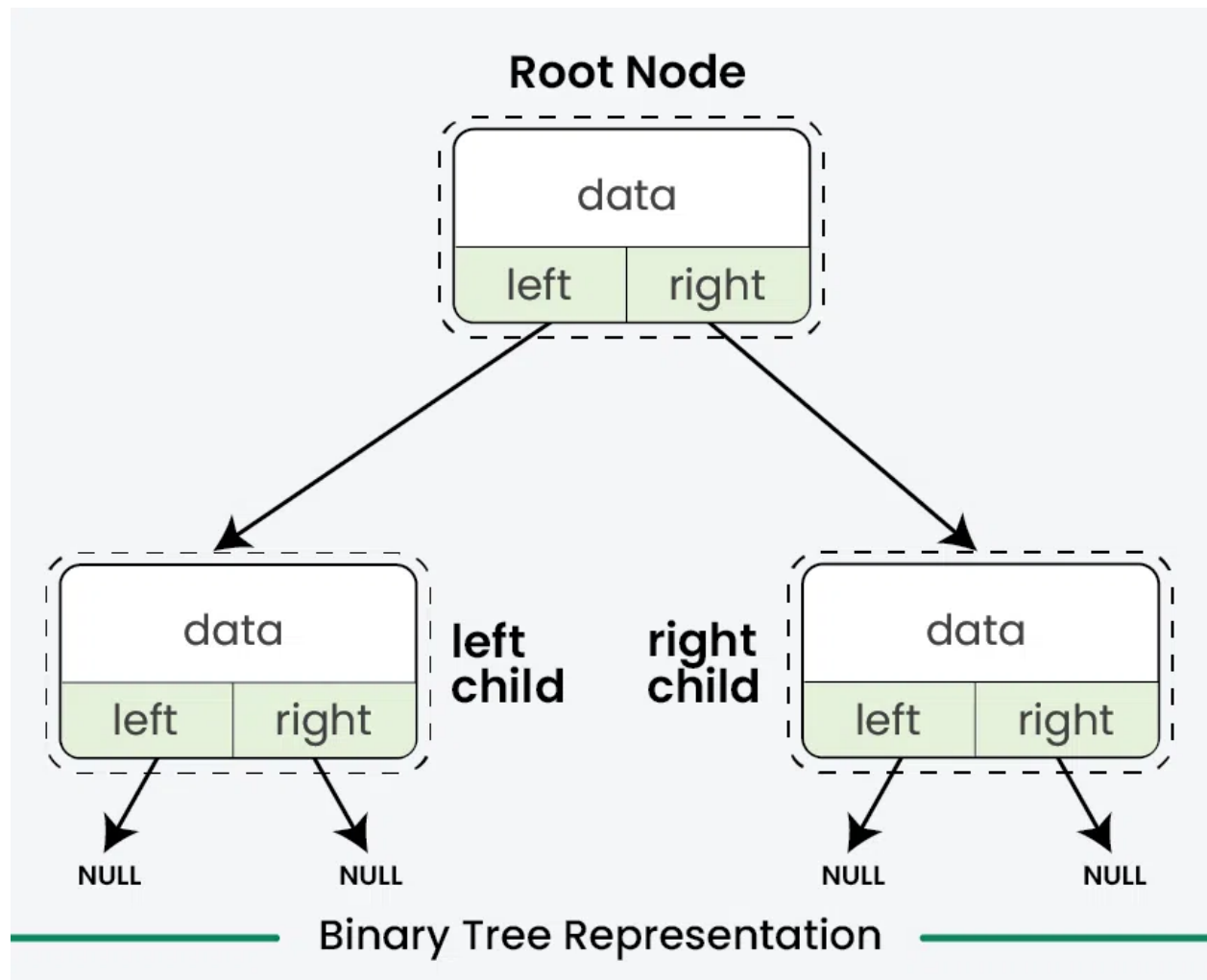
A **Binary Tree Data Structure** is a hierarchical data structure in which each node has at most two children, referred to as the left child and the right child. It is commonly used in computer science for efficient storage and retrieval of data, with various operations such as insertion, deletion, and traversal.



Representation of Binary Tree

Each node in a Binary Tree has three parts:

- Data
- Pointer to the left child
- Pointer to the right child



Create/Declare a Node of a Binary Tree

// Structure of each node of the tree.

```
struct Node {  
    int data;  
    struct Node* left;  
    struct Node* right;  
};
```

1

2

3

4

5

6

7

```
// Note : Unlike other languages, C does not support
// Object Oriented Programming. So we need to write
// a separat method for create and instance of tree node
```

```
struct Node* newNode(int item) {
    struct Node* temp =
        (struct Node*)malloc(sizeof(struct Node));
    temp->key = item;
    temp->left = temp->right = NULL;
    return temp;
}
```

Example for Creating a Binary Tree

Here's an example of creating a Binary Tree with four nodes (2, 3, 4, 5)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};
```

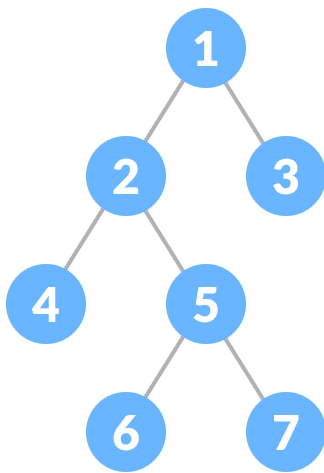
```
struct Node* createNode(int d) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = d;  
    newNode->left = NULL;  
    newNode->right = NULL;  
    return newNode;  
}
```

```
int main() {  
    // Initialize and allocate memory for tree nodes  
    struct Node* firstNode = createNode(2);  
    struct Node* secondNode = createNode(3);  
    struct Node* thirdNode = createNode(4);  
    struct Node* fourthNode = createNode(5);  
  
    // Connect binary tree nodes  
    firstNode->left = secondNode;  
    firstNode->right = thirdNode;  
    secondNode->left = fourthNode;  
  
    return 0;  
}
```

Types of Binary Tree

1. Full Binary Tree

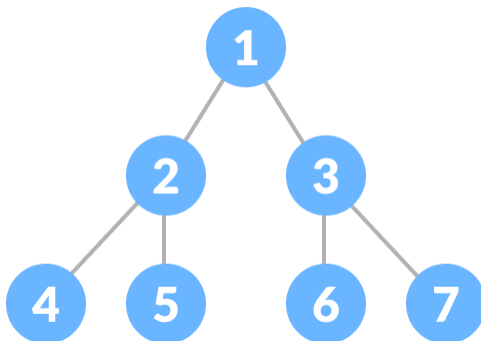
A full Binary tree is a special type of binary tree in which every parent node/internal node has either two or no children.



Full Binary Tree

2. Perfect Binary Tree

A perfect binary tree is a type of binary tree in which every internal node has exactly two child nodes and all the leaf nodes are at the same level.

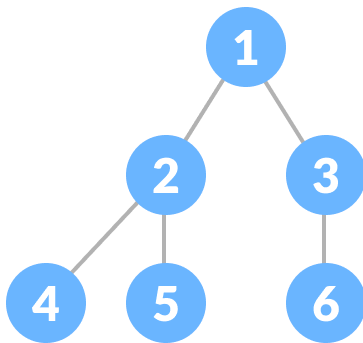


Perfect Binary Tree

3. Complete Binary Tree

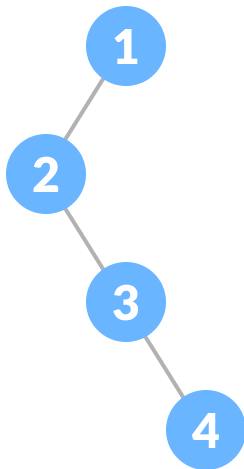
A complete binary tree is just like a full binary tree, but with two major differences

1. Every level must be completely filled
2. All the leaf elements must lean towards the left.
3. The last leaf element might not have a right sibling i.e. a complete binary tree doesn't have to be a full binary tree.



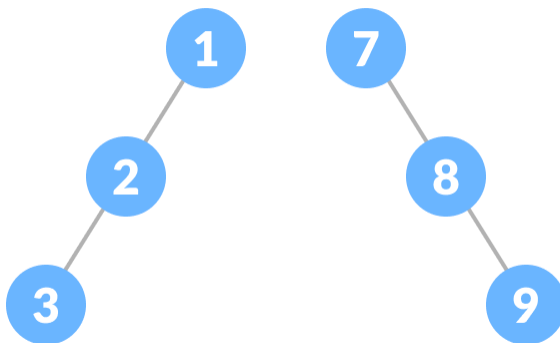
4. Degenerate or Pathological Tree

A degenerate or pathological tree is the tree having a single child either left or right.



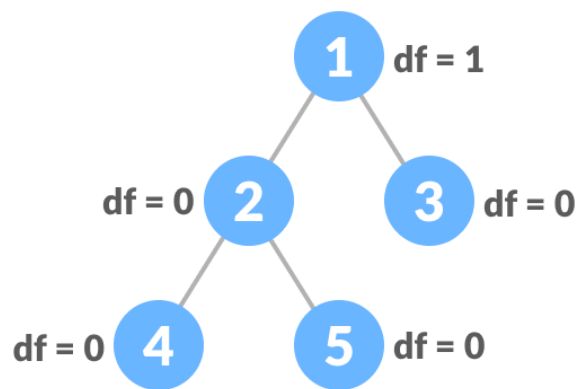
5. Skewed Binary Tree

A skewed binary tree is a pathological/degenerate tree in which the tree is either dominated by the left nodes or the right nodes. Thus, there are two types of skewed binary tree: **left-skewed binary tree** and **right-skewed binary tree**.

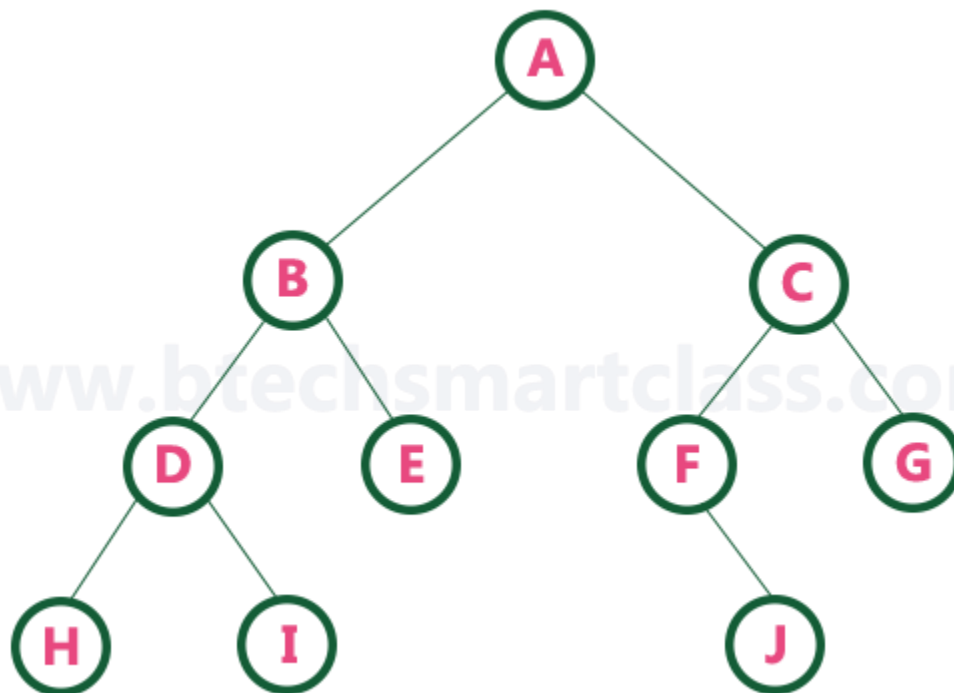


6. Balanced Binary Tree

It is a type of binary tree in which the difference between the height of the left and the right subtree for each node is either 0 or 1.



Array Representation of Binary Tree:



0 1 2 3 4 5 6 7 8 9 10 11 12

A	B	C	D	E	F	G	H	I				J
---	---	---	---	---	---	---	---	---	--	--	--	---

. If node is at ith index

- Left child would be at $[(2 * i) + 1]$.
- Right child would be at $[(2 * i) + 2]$.
- Parent would be at $[(i - 1) / 2]$

Tree Traversal Techniques

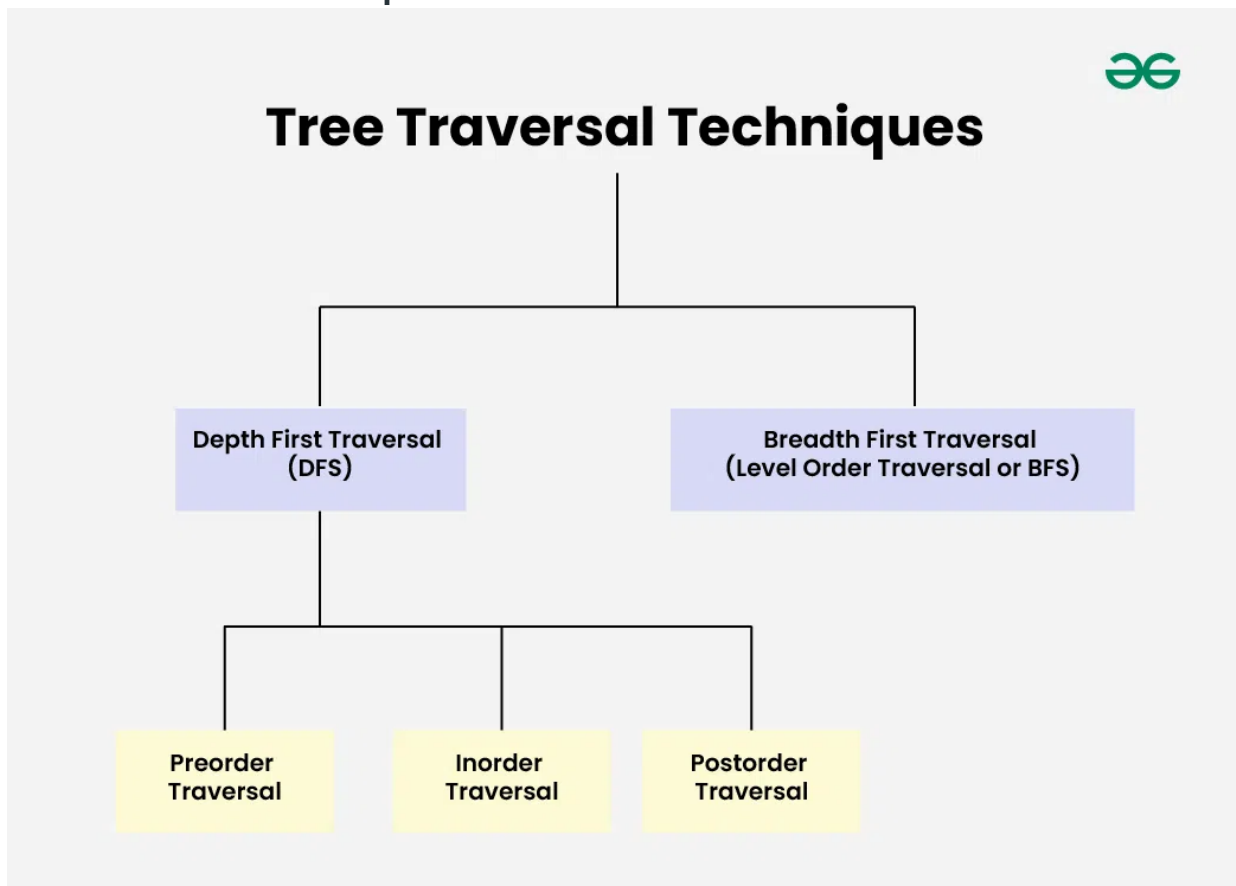
•

Tree Traversal techniques include various ways to visit all the nodes of the tree. Unlike linear data structures (Array, Linked List, Queues, Stacks, etc) which have only one logical way to traverse them, trees can be traversed in different ways. In this article, we will discuss all the tree traversal techniques along with their uses.

Tree Traversal Meaning:

Tree Traversal refers to the process of visiting or accessing each node of the tree exactly once in a certain order. Tree traversal algorithms help us to visit and process all the nodes of the tree. Since tree is not a linear data structure, there are multiple nodes which we can visit after visiting a certain node. There are multiple tree traversal techniques which decide the order in which the nodes of the tree are to be visited.

Tree Traversal Techniques:



A Tree Data Structure can be traversed in following ways:

- **Depth First Search or DFS**
 - Inorder Traversal
 - Preorder Traversal
 - Postorder Traversal
- Level Order Traversal or Breadth First Search or BFS

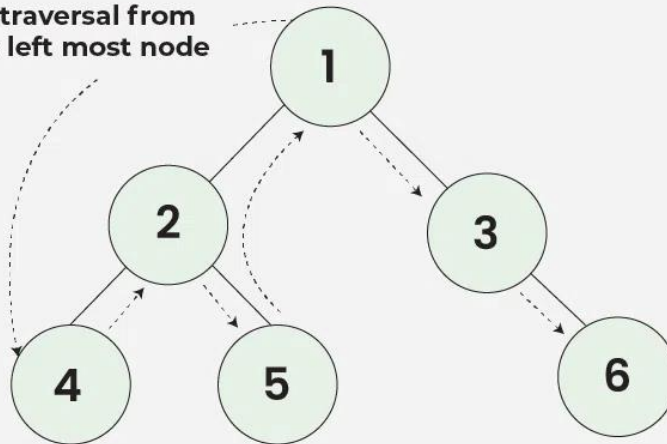
1. Inorder Traversal:

Inorder traversal visits the node in the order: **Left -> Root -> Right**



Inorder Traversal of Binary Tree

Initial traversal from
root to left most node



Inorder Traversal: $4 \rightarrow 2 \rightarrow 5 \rightarrow 1 \rightarrow 3 \rightarrow 6$

Algorithm for Inorder Traversal:

Inorder(tree)

- *Traverse the left subtree, i.e., call $\text{Inorder}(\text{left} \rightarrow \text{subtree})$*
- *Visit the root.*
- *Traverse the right subtree, i.e., call $\text{Inorder}(\text{right} \rightarrow \text{subtree})$*

Uses of Inorder Traversal:

- In the case of binary search trees (BST), Inorder traversal gives nodes in non-decreasing order (sorting order).
- To get nodes of BST in non-increasing order, a variation of Inorder traversal where Inorder traversal is reversed can be used.
- Inorder traversal can be used to evaluate arithmetic expressions stored in expression trees.

Time Complexity: $O(N)$

2. Preorder Traversal:

Preorder traversal visits the node in the order: **Root -> Left -> Right**

Algorithm for Preorder Traversal:

Preorder(tree)

- Visit the root.
- Traverse the left subtree, i.e., call *Preorder(left->subtree)*
- Traverse the right subtree, i.e., call *Preorder(right->subtree)*

Uses of Preorder Traversal:

- Preorder traversal is used to create a copy of the tree.
- Preorder traversal is also used to get prefix expressions on an expression tree.

Time Complexity: $O(N)$

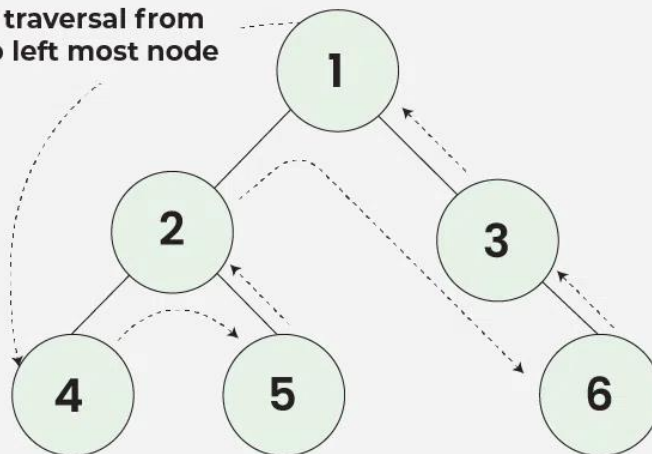
3. Postorder Traversal:

Postorder traversal visits the node in the order: **Left -> Right -> Root**

Postorder Traversal of Binary Tree



Initial traversal from
root to left most node



Postorder Traversal: $4 \rightarrow 5 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 1$

Algorithm for Postorder Traversal:

Algorithm Postorder(tree)

- *Traverse the left subtree, i.e., call Postorder(left->subtree)*
- *Traverse the right subtree, i.e., call Postorder(right->subtree)*
- *Visit the root*

Uses of Postorder Traversal:

- Postorder traversal is used to delete the tree.
- Postorder traversal is also useful to get the postfix expression of an expression tree.
- Postorder traversal can help in garbage collection algorithms, particularly in systems where manual memory management is used.

Huffman Coding

Huffman Coding is a technique of compressing data to reduce its size without losing any of the details. It was first developed by David Huffman.

Huffman Coding is generally useful to compress the data in which there are frequently occurring characters.

How Huffman Coding works?

Suppose the string below is to be sent over a network.

B	C	A	A	D	D	D	C	C	A	C	A	C	A	C
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Initial string

Each character occupies 8 bits. There are a total of 15 characters in the above string. Thus, a total of $8 * 15 = 120$ bits are required to send this string.

Using the Huffman Coding technique, we can compress the string to a smaller size.

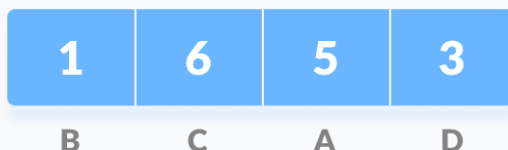
Huffman coding first creates a tree using the frequencies of the character and then generates code for each character.

Once the data is encoded, it has to be decoded. Decoding is done using the same tree.

Huffman Coding prevents any ambiguity in the decoding process using the concept of **prefix code** ie. a code associated with a character should not be present in the prefix of any other code. The tree created above helps in maintaining the property.

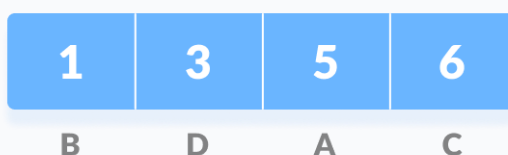
Huffman coding is done with the help of the following steps.

1. Calculate the frequency of each character in the string.



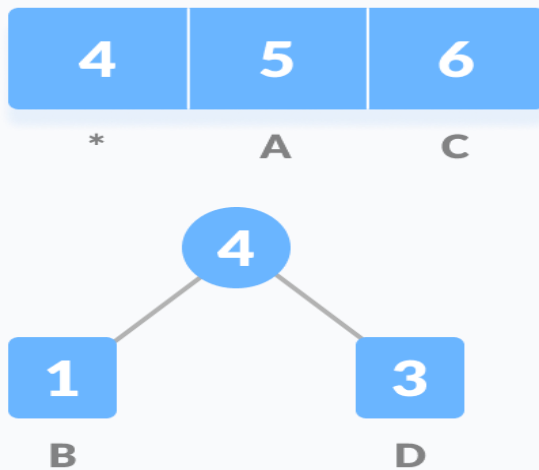
Frequency of string

2. Sort the characters in increasing order of the frequency. These are stored in a priority queue Q



Characters sorted according to the frequency

3. Make each unique character as a leaf node.
4. Create an empty node z . Assign the minimum frequency to the left child of z and assign the second minimum frequency to the right child of z . Set the value of the z as the sum of the above two minimum frequencies.

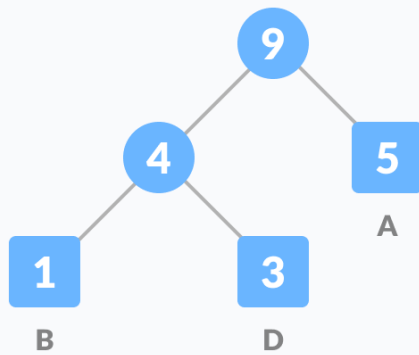


Getting the sum of the least numbers

5. Remove these two minimum frequencies from Q and add the sum into the list of frequencies (* denote the internal nodes in the figure above).
6. Insert node z into the tree.



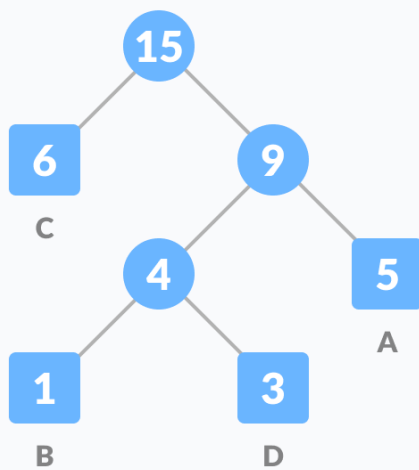
C *



Repeat steps 3 to 5 for all the characters.

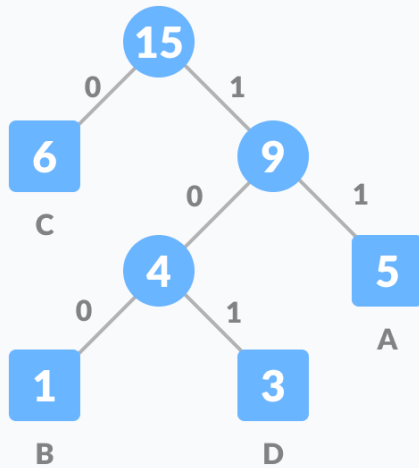


*



Repeat steps 3 to 5 for all the characters.

7. For each non-leaf node, assign 0 to the left edge and 1 to the right edge.



Assign 0 to the left edge and 1 to the right edge

For sending the above string over a network, we have to send the tree as well as the above compressed-code. The total size is given by the table below.

Character	Frequency	Code	Size
A	5	11	$5 \times 2 = 10$
B	1	100	$1 \times 3 = 3$
C	6	0	$6 \times 1 = 6$
D	3	101	$3 \times 3 = 9$

$4 * 8 = 32$ bits

15 bits

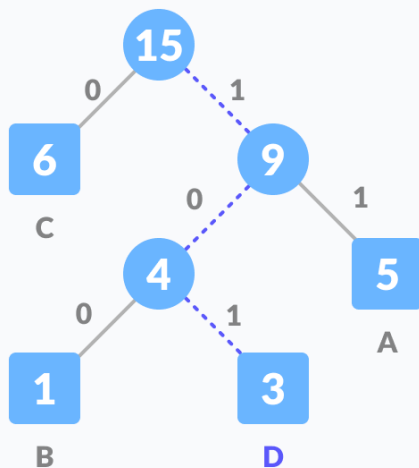
28 bits

Without encoding, the total size of the string was 120 bits. After encoding the size is reduced to $32 + 9 + 28 = 69$.

Decoding the code

For decoding the code, we can take the code and traverse through the tree to find the character.

Let 101 is to be decoded, we can traverse from the root as in the figure below.

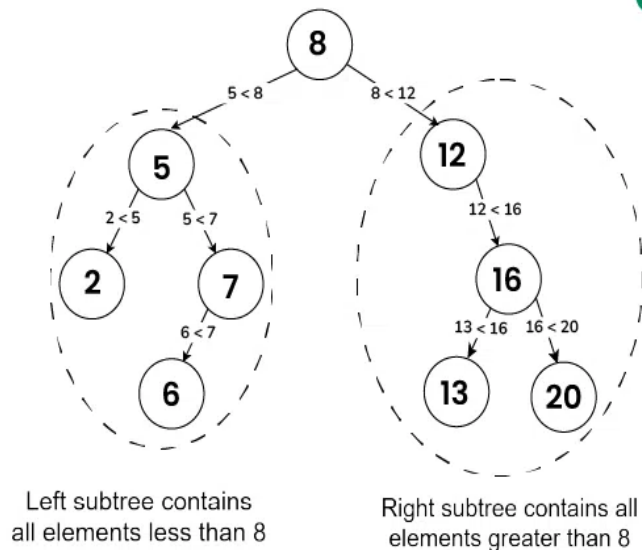


Introduction to Binary Search Tree

• **Binary Search Tree** is a data structure used in computer science for organizing and storing data in a sorted manner. Binary search tree follows all properties of binary tree and for every nodes, its **left** subtree contains values less than the node and the **right** subtree contains values greater than the node. This hierarchical structure allows for efficient **Searching, Insertion,** and **Deletion** operations on the data stored in the tree.

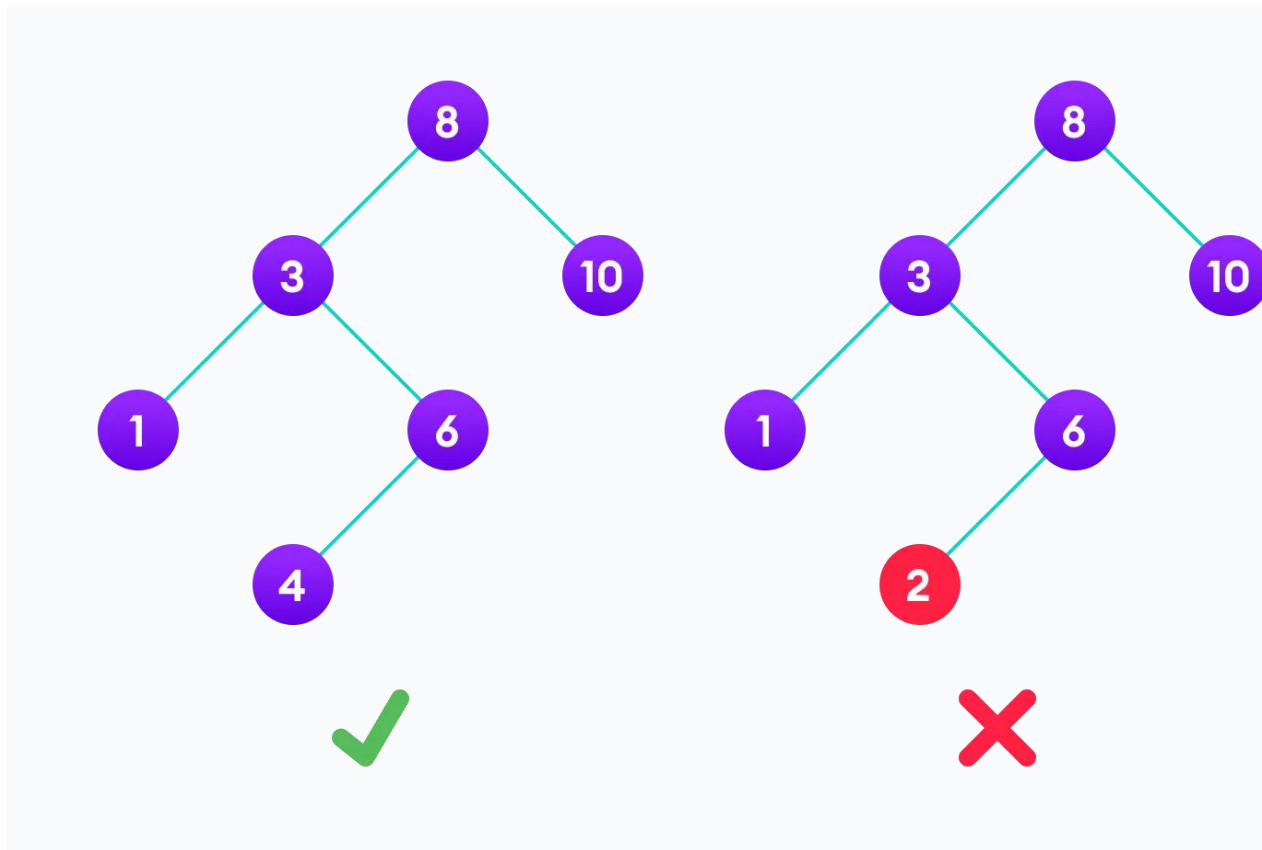


Binary Search Tree



The properties of BST

1. All nodes of left subtree are less than the root node
2. All nodes of right subtree are more than the root node
3. Both subtrees of each node are also BSTs i.e. they have the above two properties



Algorithm: Inserting a Node into a Binary Search Tree (BST)

Input:

- A pointer to the root of the BST (root).
- The value to be inserted (value).

Output:

- The modified BST with the new node inserted.

Steps:

1. **Base Case:**

- If the tree is empty (root == NULL):
 1. Dynamically allocate memory for a new node.
 2. Assign value to the new node's data.
 3. Set the left and right child pointers of the new node to NULL.

4. Return the new node.

2. Recursive Case:

- If the value to be inserted is **less than** the current node's value (root->data):
 - Recursively call the insertion function for the **left subtree**.
 - Assign the result of the recursive call to root->left.
- Else if the value is **greater than** the current node's value (root->data):
 - Recursively call the insertion function for the **right subtree**.
 - Assign the result of the recursive call to root->right.

3. Return:

- Return the root of the tree.

Insert Operation

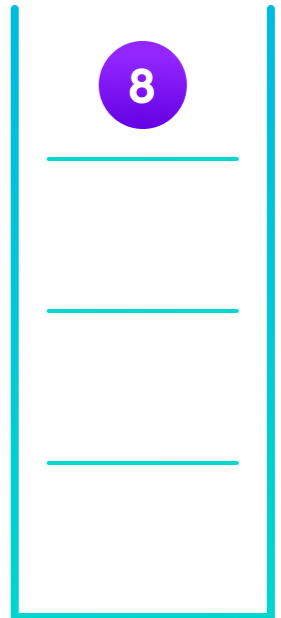
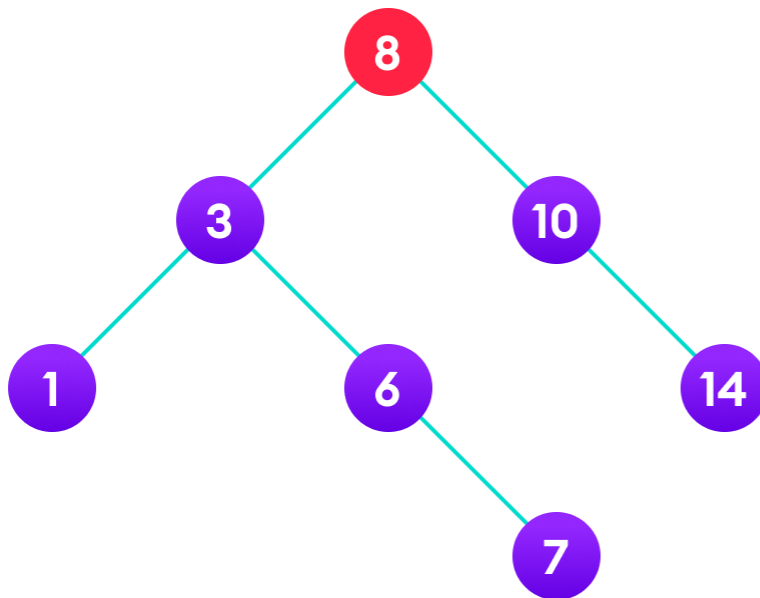
Inserting a value in the correct position is similar to searching because we try to maintain the rule that the left subtree is lesser than root and the right subtree is larger than root.

We keep going to either right subtree or left subtree depending on the value and when we reach a point left or right subtree is null, we put the new node there.

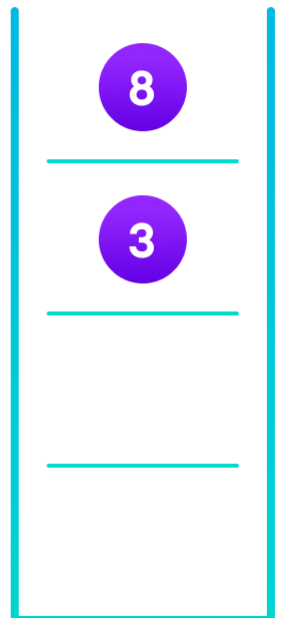
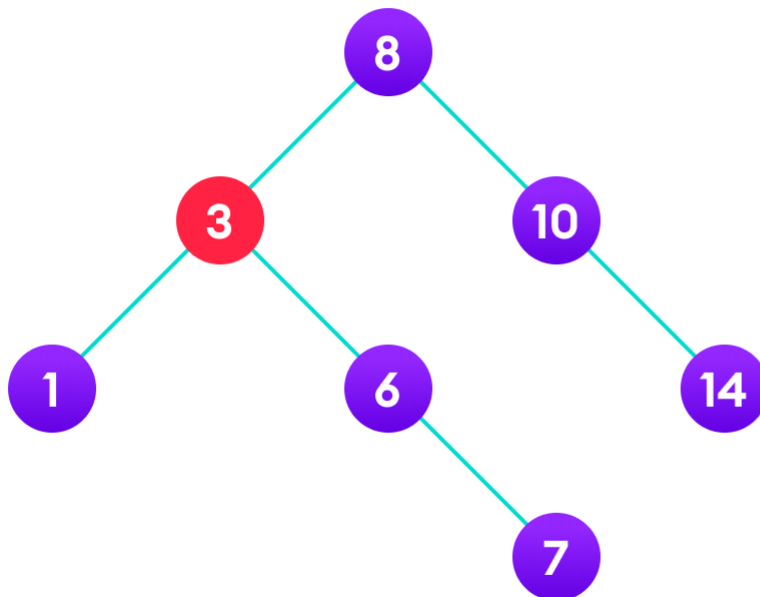
Algorithm:

```
If node == NULL
    return createNode(data)
if (data < node->data)
    node->left = insert(node->left, data);
else if (data > node->data)
    node->right = insert(node->right, data);
return node;
```

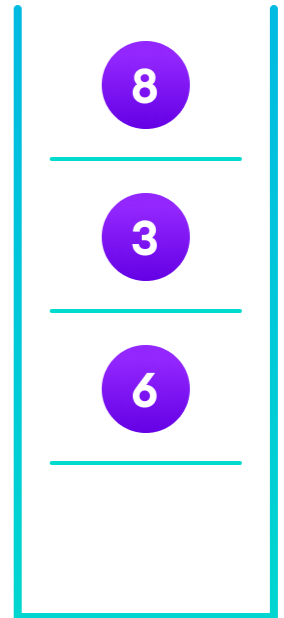
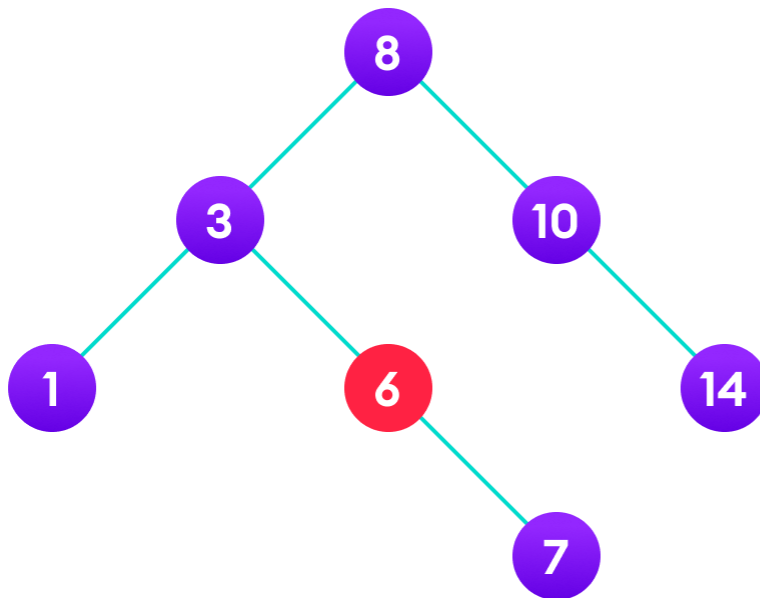
The algorithm isn't as simple as it looks. Let's try to visualize how we add a number to an existing BST.



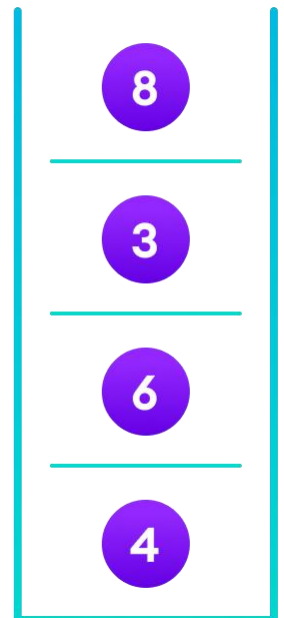
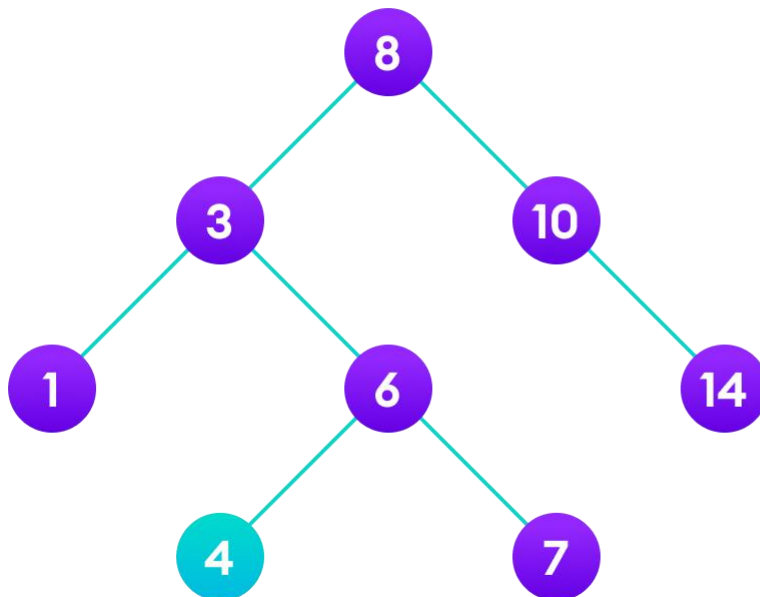
4<8 so, transverse through the left child of 8



4>3 so, transverse through the right child of 8



4<6 so, transverse through the left child of 6



Insert 4 as a left child of 6

Algorithm: Deleting a Node into a Binary Search Tree (BST)

Input:

- A pointer to the root of the BST (root).
- The value to be deleted (value).

Output:

- The modified BST after deleting the node with the specified value.

Steps:

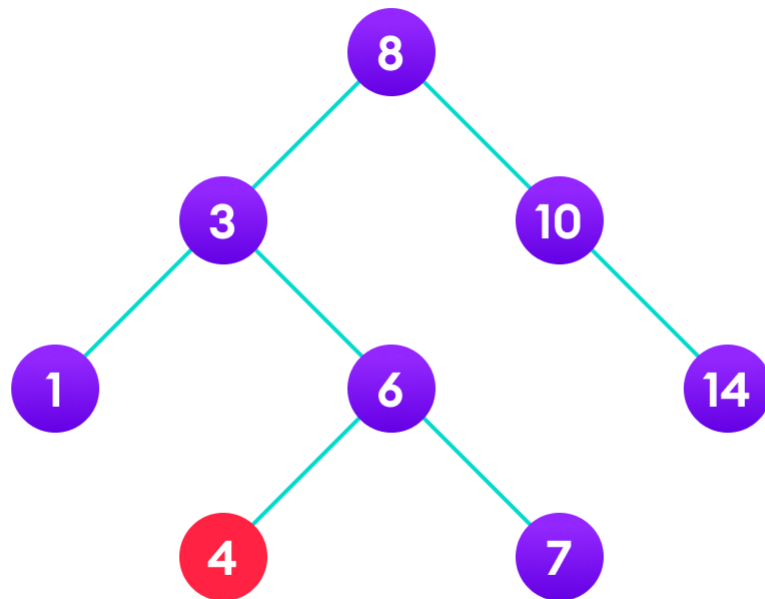
1. **Base Case:**
 - If the tree is empty (root == NULL), return NULL.
2. **Traverse to Find the Node:**
 - If value < root->data, recursively call the deletion function for the **left subtree**.
 - Else if value > root->data, recursively call the deletion function for the **right subtree**.
3. **Node to Delete Found:**
 - If value == root->data:
 - **Case 1:** Node has no children (leaf node).
 - Free the node and return NULL.
 - **Case 2:** Node has only one child.
 - Replace the node with its child (left or right), free the node, and return the new child.
 - **Case 3:** Node has two children.
 - Find the **in-order successor** (smallest value in the right subtree).
 - Replace the node's value with the in-order successor's value.
 - Recursively delete the in-order successor.
4. **Return the Root:**
 - Return the updated root after deletion.

Deletion Operation

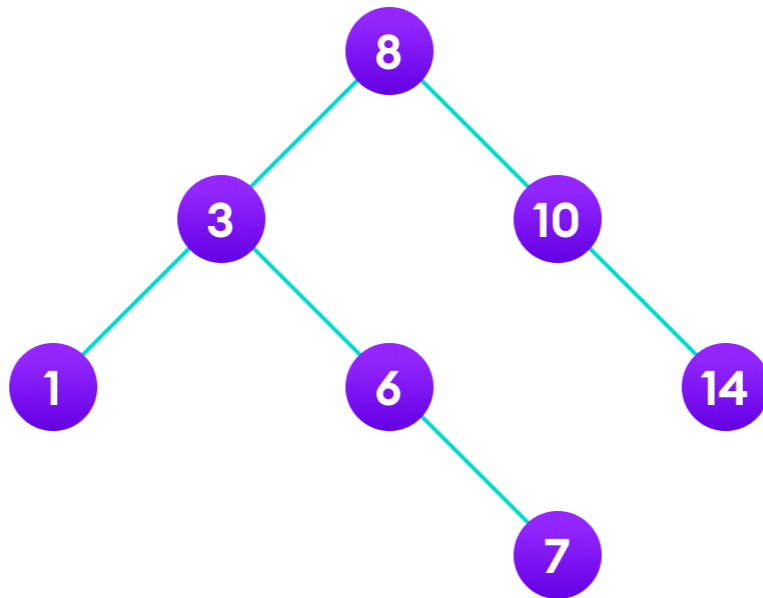
There are three cases for deleting a node from a binary search tree.

Case I

In the first case, the node to be deleted is the leaf node. In such a case, simply delete the node from the tree.



4 is to be deleted

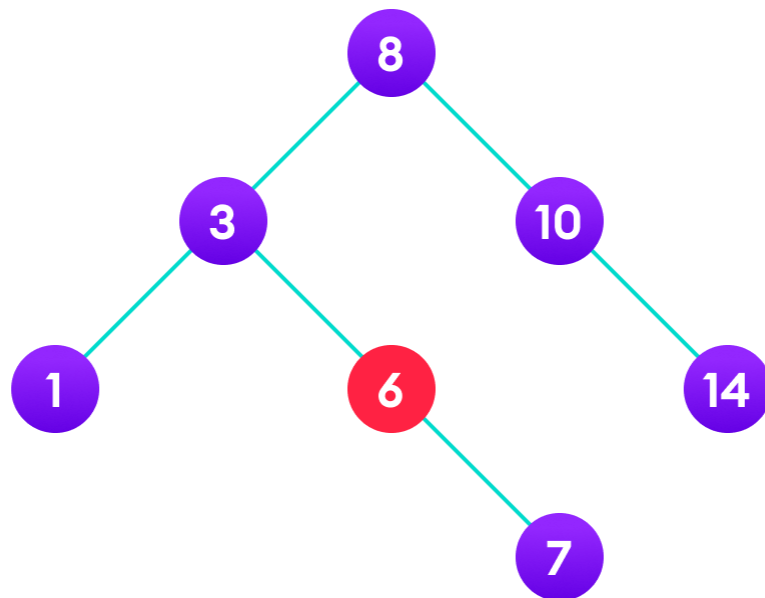


Delete the node

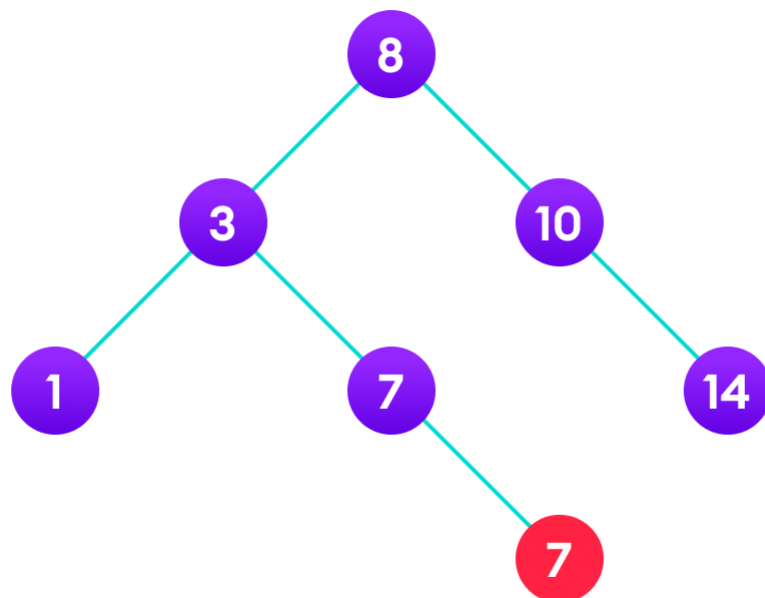
Case II

In the second case, the node to be deleted lies has a single child node. In such a case follow the steps below:

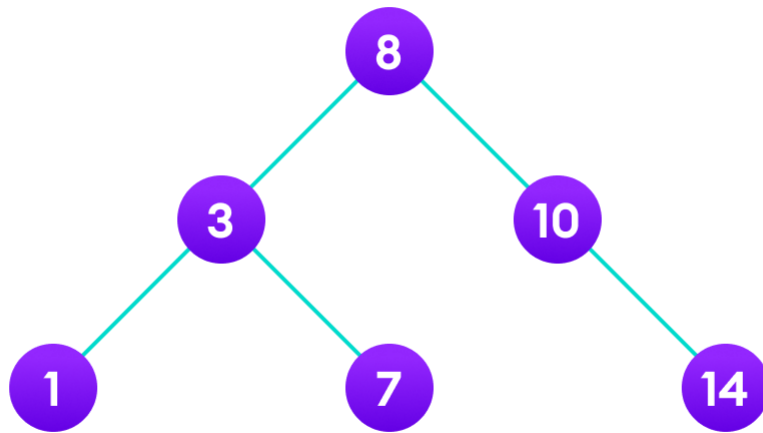
1. Replace that node with its child node.
2. Remove the child node from its original position.



6 is to be deleted



copy the value of its child to the node and delete the child

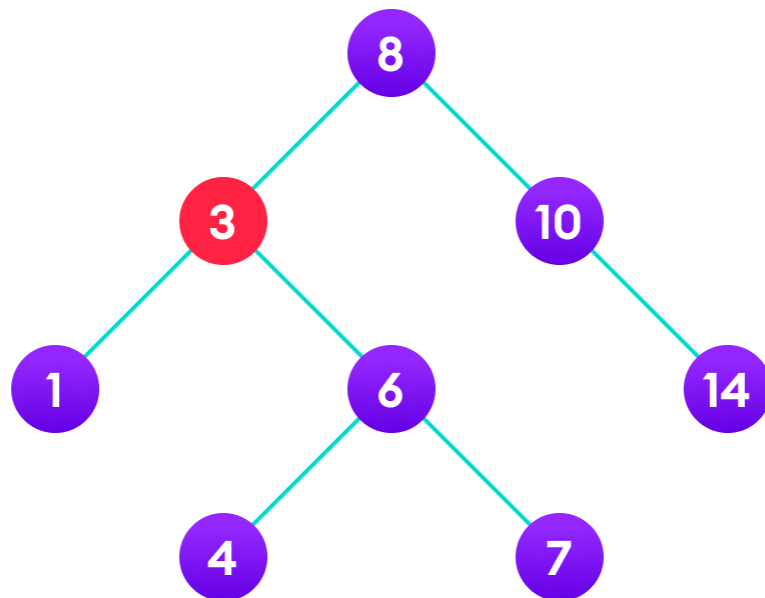


Final tree

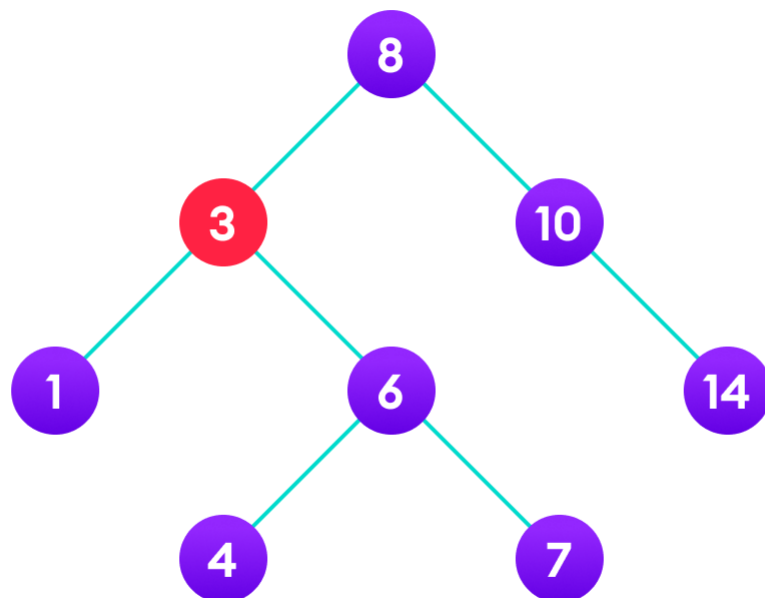
Case III

In the third case, the node to be deleted has two children. In such a case follow the steps below:

1. Get the inorder successor of that node.
2. Replace the node with the inorder successor.
3. Remove the inorder successor from its original position.
4. Or remove the inorder predecessors from its original position.



3 is to be deleted



Copy the value of the inorder successor (4) to the node

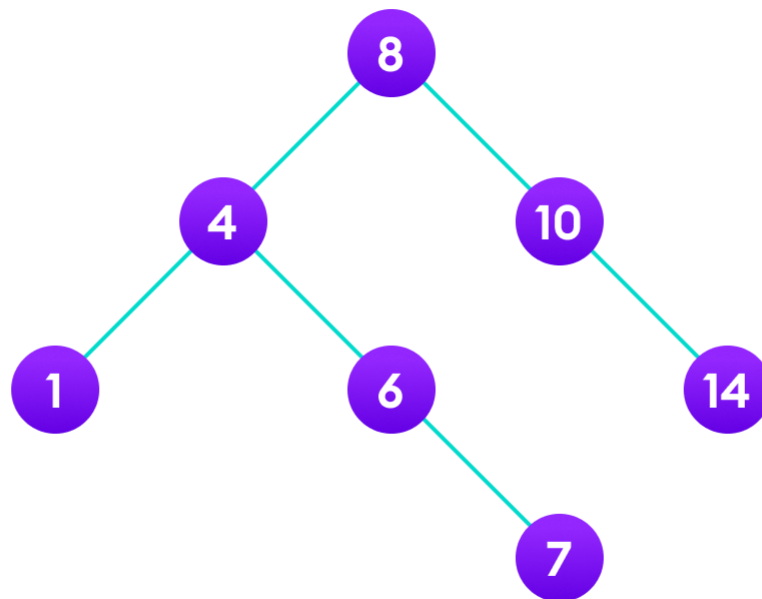


fig : Delete the inorder successor

Algorithm: Searching a Node into a Binary Search Tree (BST)

Input:

- A pointer to the root of the BST (root).
- The value to be searched (value).

Output:

- A pointer to the node containing the value, or NULL if the value is not found.

Steps:

1. Base Case:

- If the tree is empty ($\text{root} == \text{NULL}$) or the value is found ($\text{root->data} == \text{value}$), return the current root.

2. Traverse the Tree:

- If $\text{value} < \text{root->data}$, recursively search in the **left subtree**.

- Else if value > root->data, recursively search in the **right subtree`.

3. Return the Result:

- Return the node if found, or NULL if not found.

Search Operation

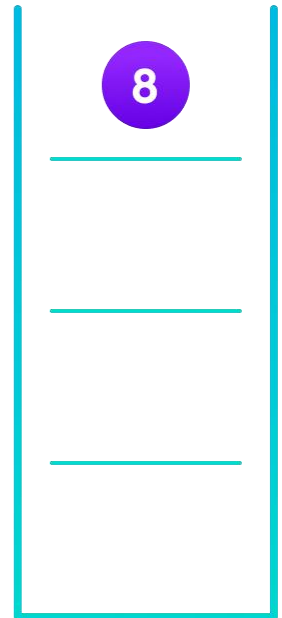
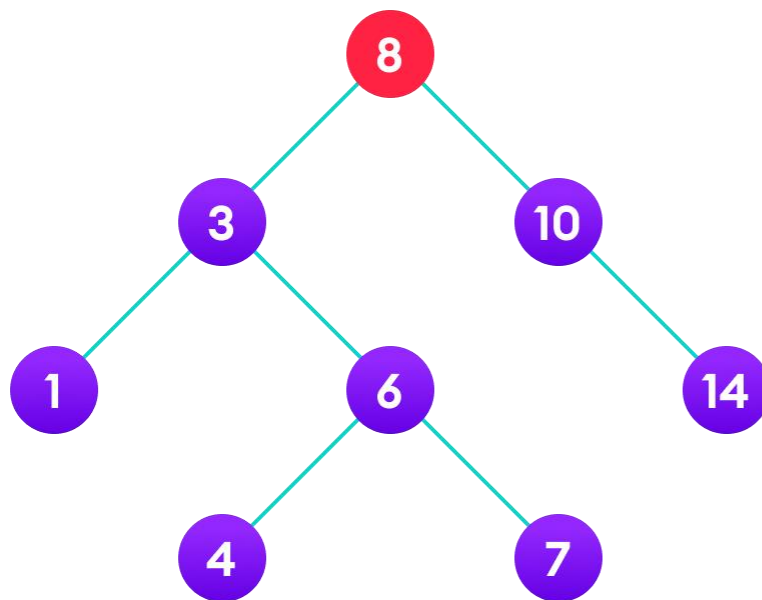
The algorithm depends on the property of BST that if each left subtree has values below root and each right subtree has values above the root.

If the value is below the root, we can say for sure that the value is not in the right subtree; we need to only search in the left subtree and if the value is above the root, we can say for sure that the value is not in the left subtree; we need to only search in the right subtree.

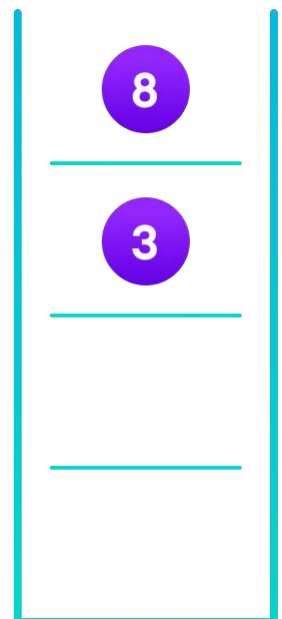
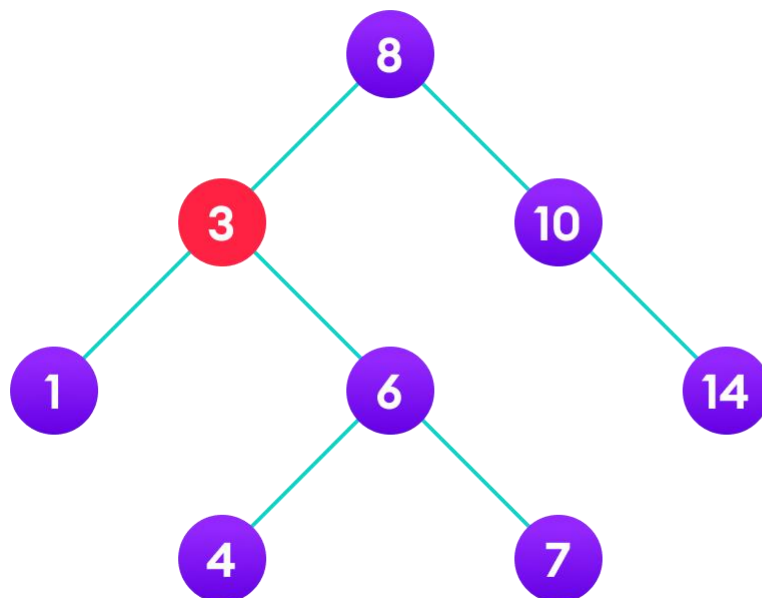
Algorithm:

```
If root == NULL
    return NULL;
If number == root->data
    return root->data;
If number < root->data
    return search(root->left)
If number > root->data
    return search(root->right)
```

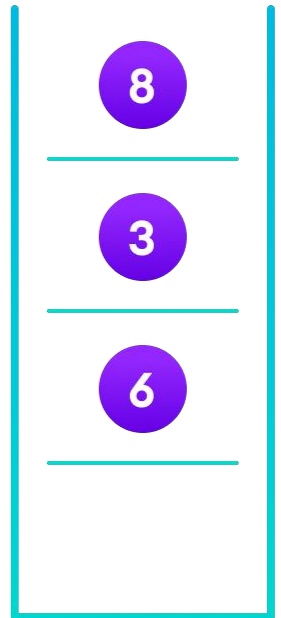
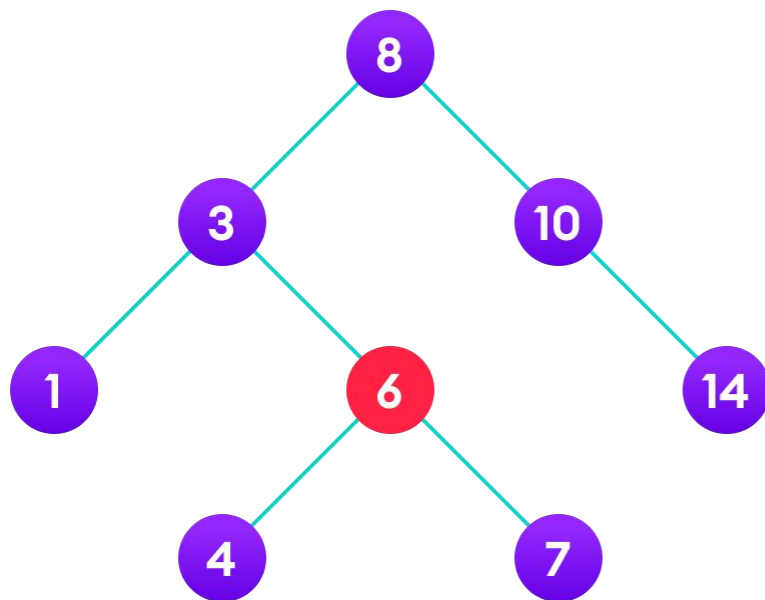
Let us try to visualize this with a diagram.



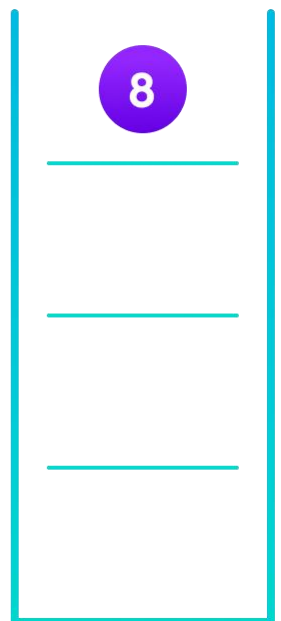
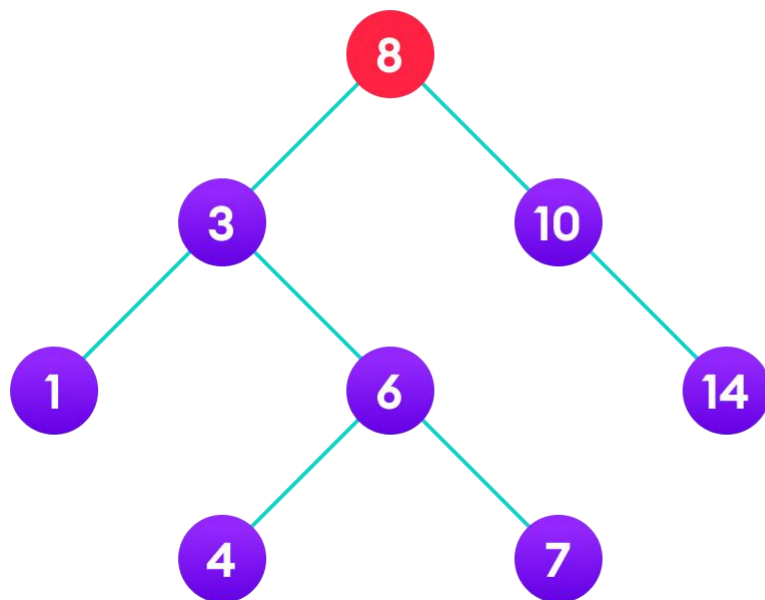
4 is not found so, traverse through the left subtree of 8



4 is not found so, traverse through the right subtree of 3



4 is not found so, traverse through the left subtree of 6



4 is found

If the value is found, we return the value so that it gets propagated in each recursion step as shown in the image below.

Algorithm: Traversing a Node into a Binary Search Tree (BST)

a. In-order Traversal (Left, Root, Right):

- Input: Pointer to the root of the BST (root).
- Output: Nodes of the BST printed in sorted order.

Steps:

1. If root is not NULL:
 - Recursively traverse the **left subtree**.
 - Print the current node's value (root->data).
 - Recursively traverse the **right subtree**.

b. Pre-order Traversal (Root, Left, Right):

- Input: Pointer to the root of the BST (root).
- Output: Nodes of the BST printed in pre-order sequence.

Steps:

1. If root is not NULL:
 - Print the current node's value (root->data).
 - Recursively traverse the **left subtree**.
 - Recursively traverse the **right subtree**.

c. Post-order Traversal (Left, Right, Root):

- Input: Pointer to the root of the BST (root).
- Output: Nodes of the BST printed in post-order sequence.

Steps:

1. If root is not NULL:

- Recursively traverse the **left subtree**.
- Recursively traverse the **right subtree**.
- Print the current node's value (root->data).

Binary Search Tree Complexities

Time Complexity

Operation	Best Case Complexity	Average Case Complexity	Worst Case Complexity
Search	$O(\log n)$	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(\log n)$	$O(n)$

Here, n is the number of nodes in the tree.

Space Complexity

The space complexity for all the operations is $O(n)$.

Construction of Binary Search Tree:

Example-

Construct a BST

50,70,60,20,90,10,40,100

When elements are given in sequence,

- Always consider the first element as the root node.
- Consider the given elements and insert them in the BST one by one/

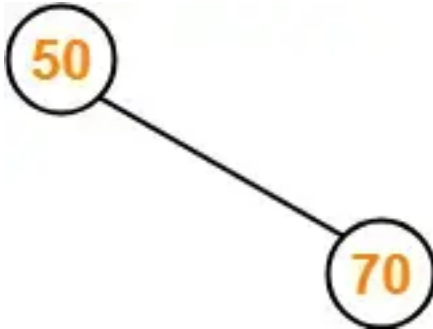
The BST will be constructed as :

Insert 50 :



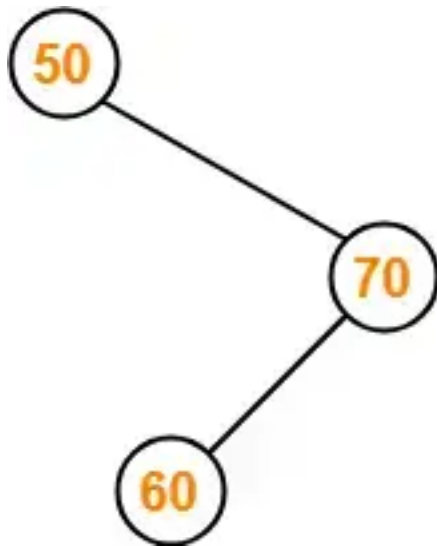
Insert 70:

- As $70 > 50$, so insert 70 to the right of 50.



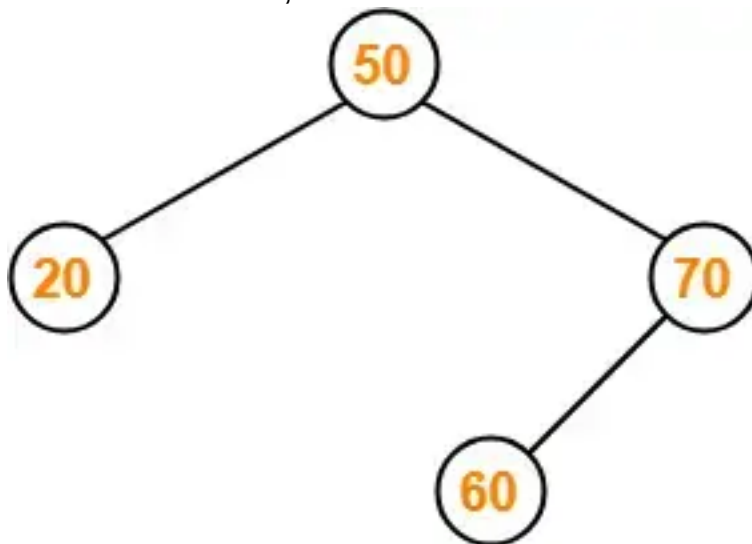
Insert 60:

- As $60 > 50$, so insert 60 to the right of 50.
- As $60 < 70$, so insert 60 to the left of 70.



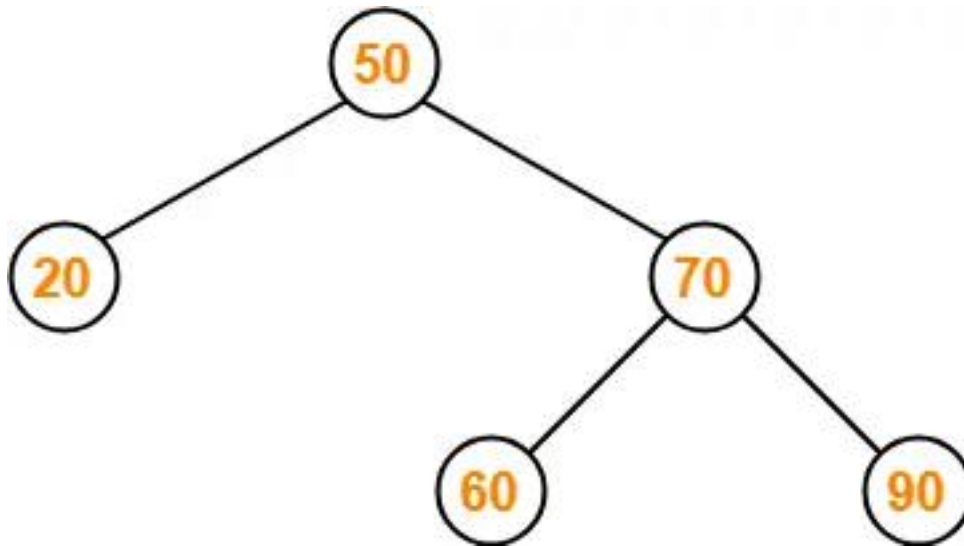
Insert 20:

- As $20 < 50$, so insert 20 to the left of 50.



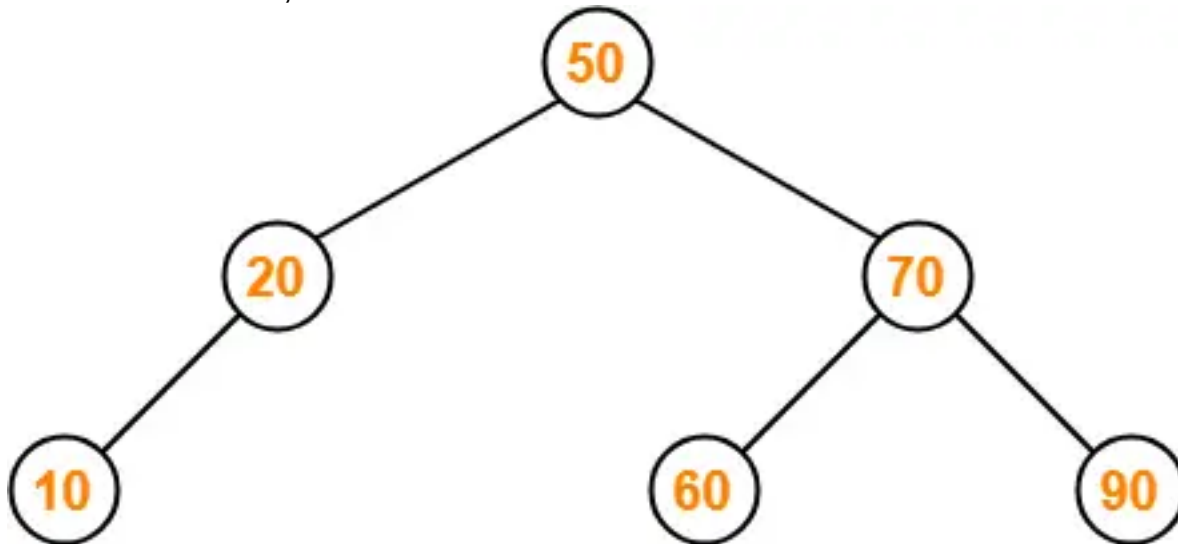
Insert 90:

- As $90 > 50$, so insert 90 to the right of 50.
- As $90 > 70$, so insert 90 to the right of 70.



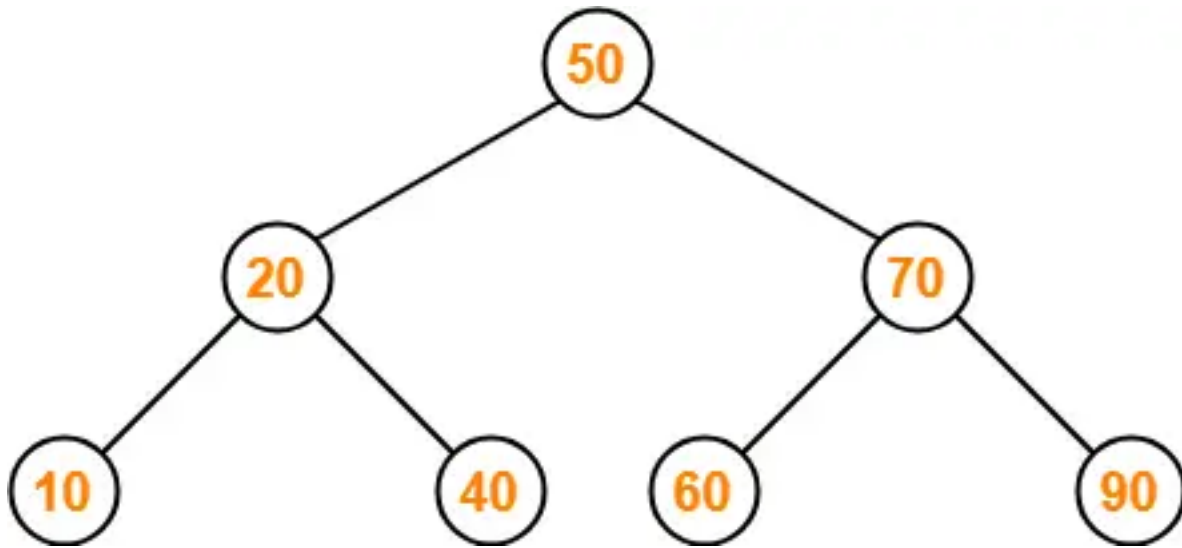
Insert 10:

- As $10 < 50$, so insert 10 to the left of 50.
- As $10 < 20$, so insert 10 to the left of 20.



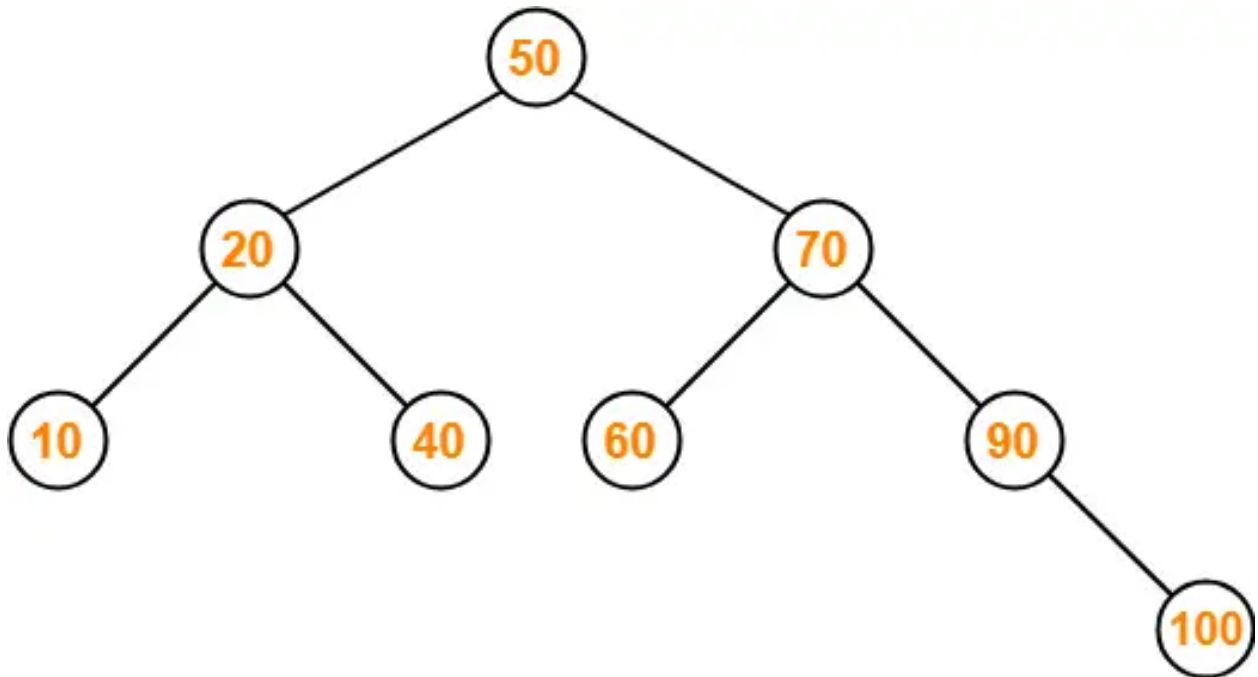
Insert 40:

- As $40 < 50$, so insert 40 to the left of 50.
- As $40 > 20$, so insert 40 to the right of 20.



Insert 100:

- As $100 > 50$, so insert 100 to the right of 50.
- As $100 > 70$, so insert 100 to the right of 70.
- As $100 > 90$, so insert 100 to the right of 90.



Binary Search Tree

Problems with Unbalanced Binary Trees:

1. Degraded Performance

- **Worst-Case Time Complexity:** An unbalanced binary tree can resemble a linked list if nodes are inserted in a specific order (e.g., in ascending or descending order). In such cases:
 - **Search, Insert, Delete operations:** Instead of the typical $O(\log n)$ time complexity for balanced binary trees, these operations degrade to $O(n)$ in unbalanced trees.

2. Inefficient Memory Utilization

- Nodes in an unbalanced binary tree are not evenly distributed, leading to memory inefficiency. Some branches may be deep with many nodes, while others may be shallow or empty.

3. Skewed Tree Structure

- An unbalanced tree might have an uneven distribution of nodes:
 - **Left-heavy or right-heavy** trees lead to inefficient traversal since one side of the tree becomes significantly deeper than the other.

4. Higher Risk of Stack Overflow

- Recursive tree traversals (e.g., in-order, pre-order, or post-order traversals) require additional memory for function call stacks. In highly unbalanced trees, deep recursion can lead to a stack overflow error.

5. Reduced Search Efficiency

- The purpose of using a binary search tree (BST) is to facilitate efficient searches. However, in an unbalanced tree, searching becomes inefficient as the search path may involve traversing nearly all nodes in the tree.

6. Difficulties in Tree Traversals

- Tree traversal operations like breadth-first search (BFS) or depth-first search (DFS) can become less predictable in their performance due to the imbalance, especially when implementing iterative versions of these algorithms.

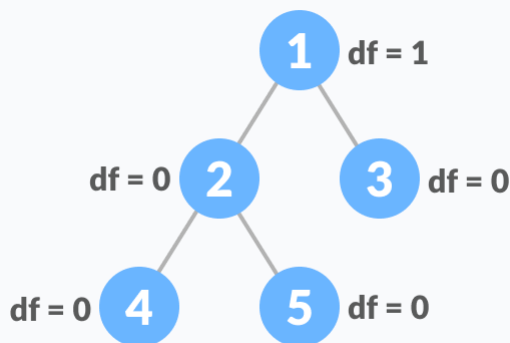
Balanced Binary Tree

A balanced binary tree, also referred to as a height-balanced binary tree, is defined as a binary tree in which the height of the left and right subtree of any node differ by not more than 1.

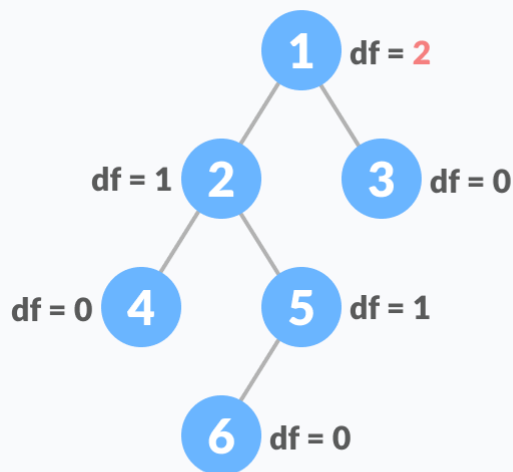
To learn more about the height of a tree/node, visit [Tree Data Structure](#).

Following are the conditions for a height-balanced binary tree:

1. Difference between the left and the right subtree for any node is not more than one
2. the left subtree is balanced
3. the right subtree is balanced



Balanced Binary Tree with depth at each level



$df = |\text{height of left child} - \text{height of right child}|$

Unbalanced Binary Tree with depth at each level

AVL Tree

AVL tree is a self-balancing binary search tree in which each node maintains extra information called a balance factor whose value is either -1, 0 or +1.

AVL tree got its name after its inventor Georgy Adelson-Velsky and Landis.

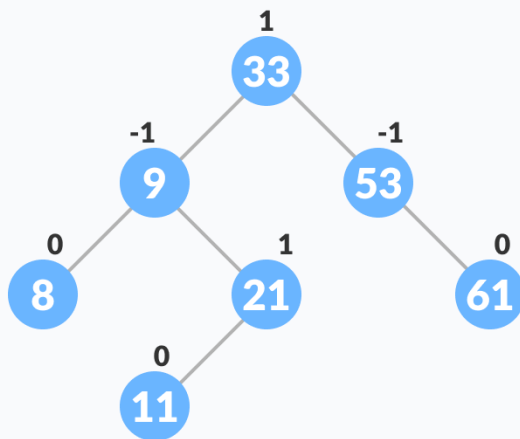
Balance Factor

Balance factor of a node in an AVL tree is the difference between the height of the left subtree and that of the right subtree of that node.

Balance Factor = (Height of Left Subtree - Height of Right Subtree) or (Height of Right Subtree - Height of Left Subtree)

The self balancing property of an avl tree is maintained by the balance factor. The value of balance factor should always be -1, 0 or +1.

An example of a balanced avl tree is:



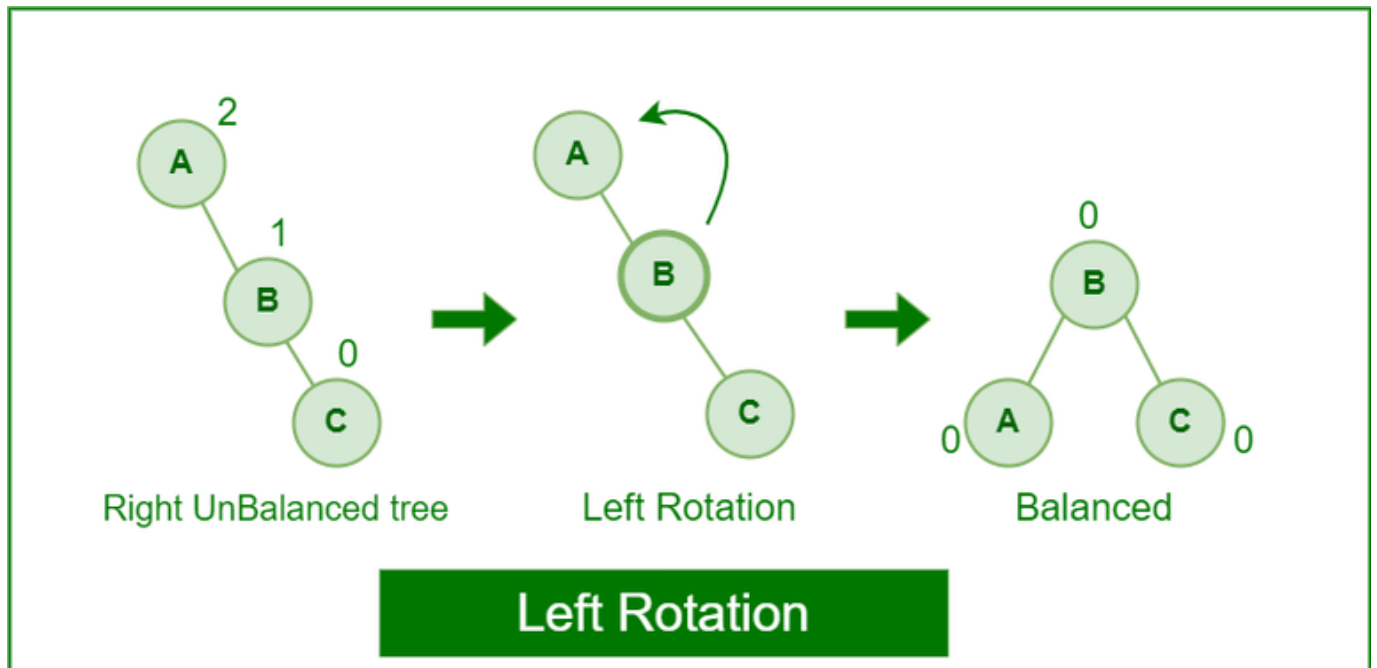
Avl tree

Rotating the subtrees in an AVL Tree:

An AVL tree may rotate in one of the following four ways to keep itself balanced:

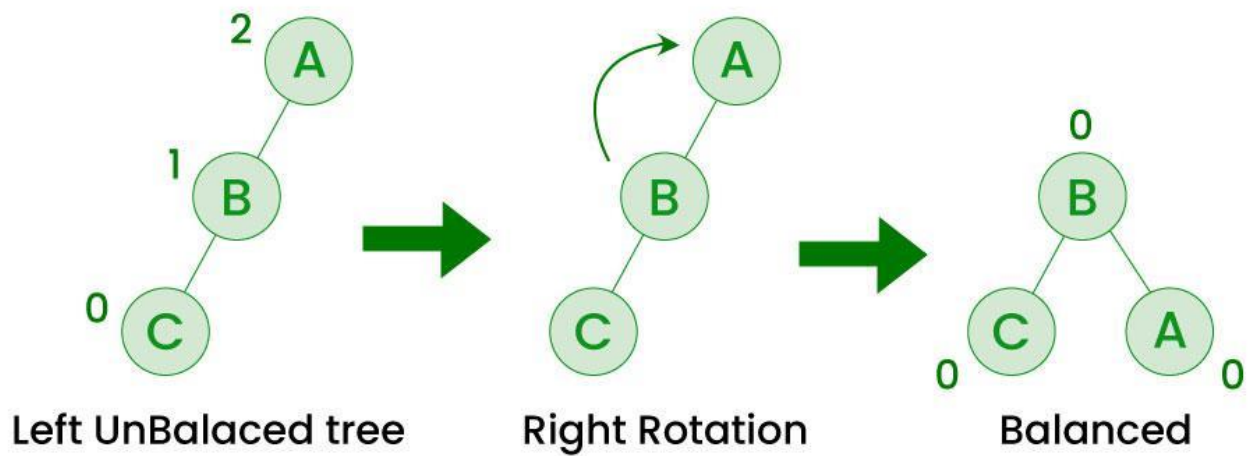
Left Rotation:

When a node is added into the right subtree of the right subtree, if the tree gets out of balance, we do a single left rotation.



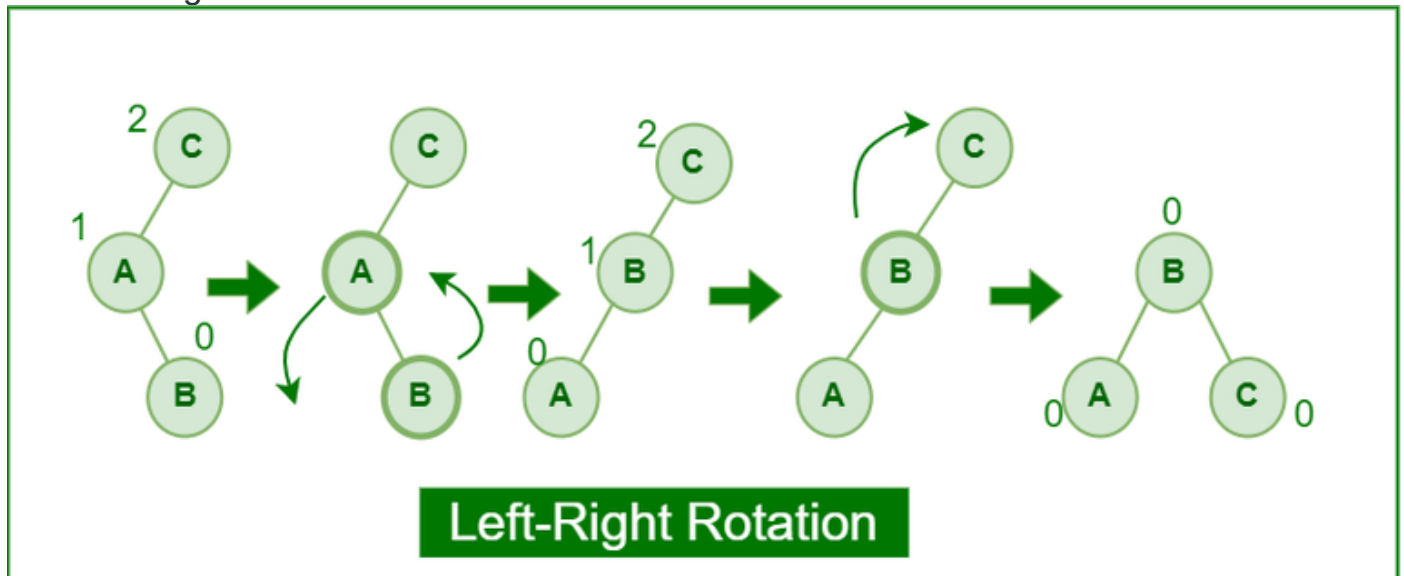
Right Rotation:

If a node is added to the left subtree of the left subtree, the AVL tree may get out of balance, we do a single right rotation.



Left-Right Rotation:

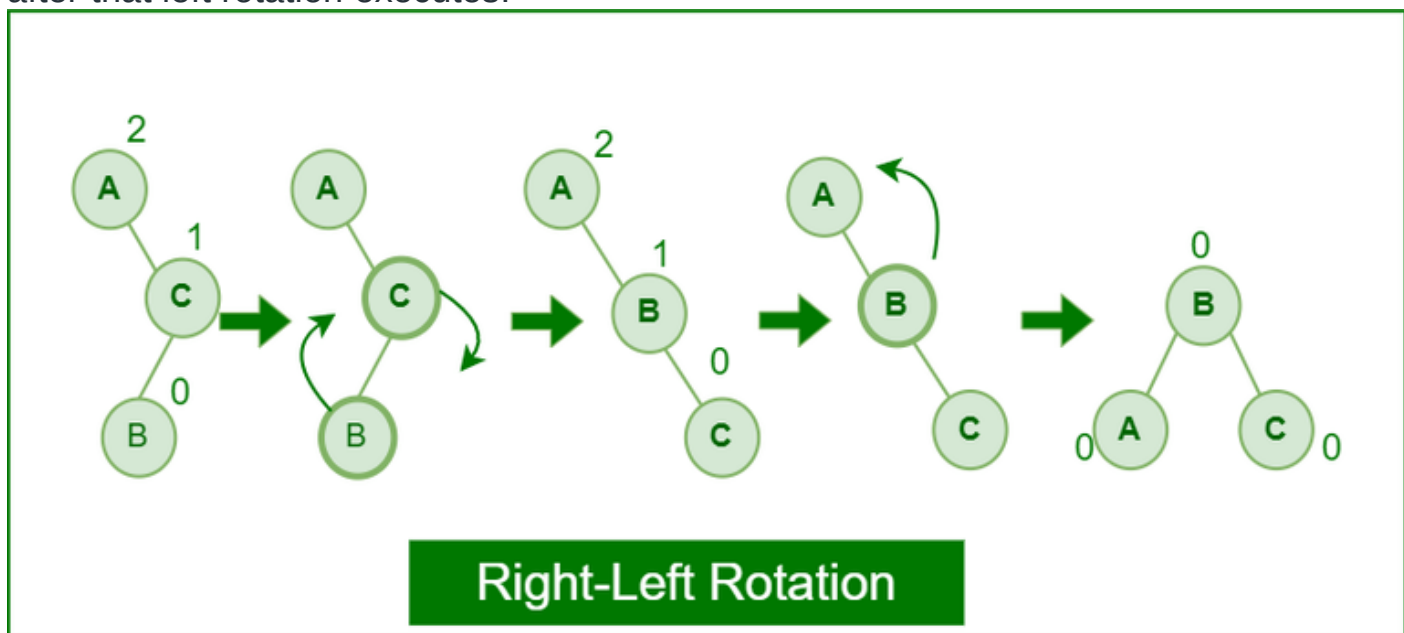
A left-right rotation is a combination in which first left rotation takes place after that right rotation executes.



Left-Right Rotation in AVL tree

Right-Left Rotation:

A right-left rotation is a combination in which first right rotation takes place after that left rotation executes.



Right-Left Rotation in AVL tree

Insertion in AVL Tree:

Steps for Insertion in an AVL Tree

1. Perform Standard BST Insertion

- Insert the key as in a regular binary search tree (BST).

2. Update Heights

- Update the height of all ancestor nodes of the newly inserted node:
 $\text{height} = 1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$

3. Calculate the Balance Factor

- For each ancestor node, calculate:
 $\text{balance factor} = \text{height}(\text{left}) - \text{height}(\text{right})$

4. Identify Imbalance Cases

- **Left-Left (LL):** Balance factor >1 , and new key is in the left subtree of the left child.
- **Right-Right (RR):** Balance factor <-1 , and new key is in the right subtree of the right child.
- **Left-Right (LR):** Balance factor >1 , and new key is in the right subtree of the left child.
- **Right-Left (RL):** Balance factor <-1 , and new key is in the left subtree of the right child.

5. Perform Rotations

- **LL:** Perform a **Right Rotation**.
- **RR:** Perform a **Left Rotation**.
- **LR:** Perform a **Left Rotation** on the left child, followed by a **Right Rotation**.
- **RL:** Perform a **Right Rotation** on the right child, followed by a **Left Rotation**.

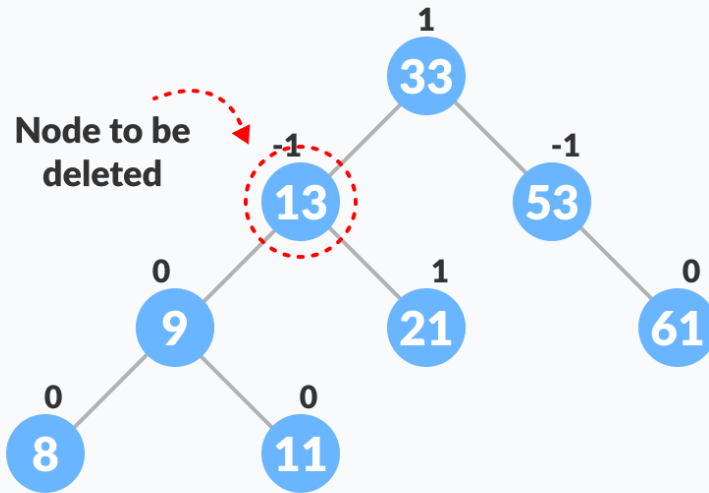
6. Return Updated Root

- After rebalancing, return the updated root of the AVL tree.

Algorithm to Delete a node

A node is always deleted as a leaf node. After deleting a node, the balance factors of the nodes get changed. In order to rebalance the balance factor, suitable rotations are performed.

1. Locate `nodeToBeDeleted` (recursion is used to find `nodeToBeDeleted` in the code



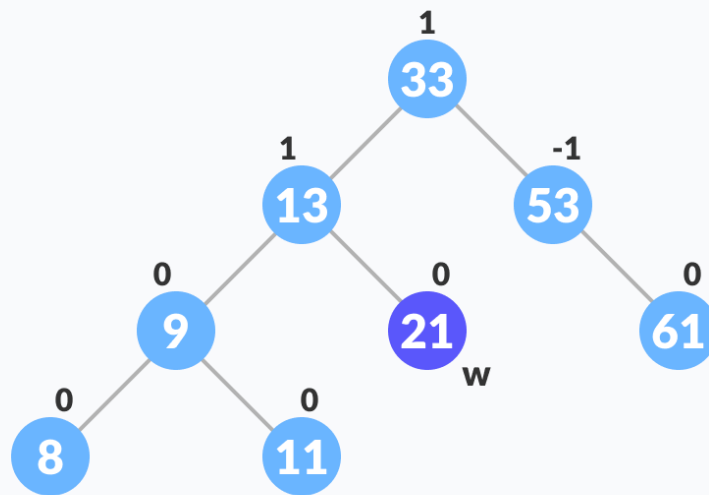
used below).

Locating the

node to be deleted

2. There are three cases for deleting a node:

- If `nodeToBeDeleted` is the leaf node (ie. does not have any child), then remove `nodeToBeDeleted`.
- If `nodeToBeDeleted` has one child, then substitute the contents of `nodeToBeDeleted` with that of the child. Remove the child.
- If `nodeToBeDeleted` has two children, find the inorder successor `w` of `nodeToBeDeleted` (ie. node with a minimum value of key in the



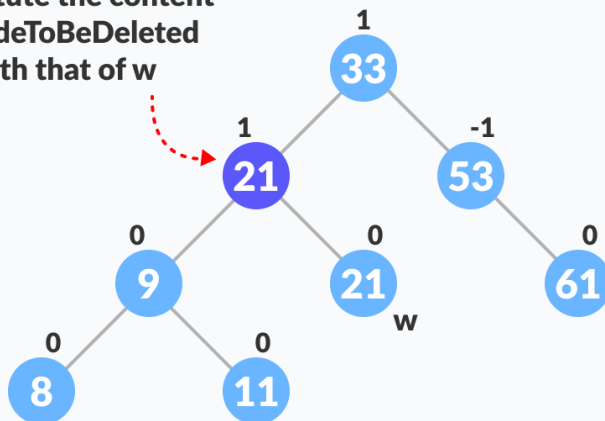
right subtree).

successor

Finding the

- a. Substitute the contents of `nodeToBeDeleted` with that of `w`.

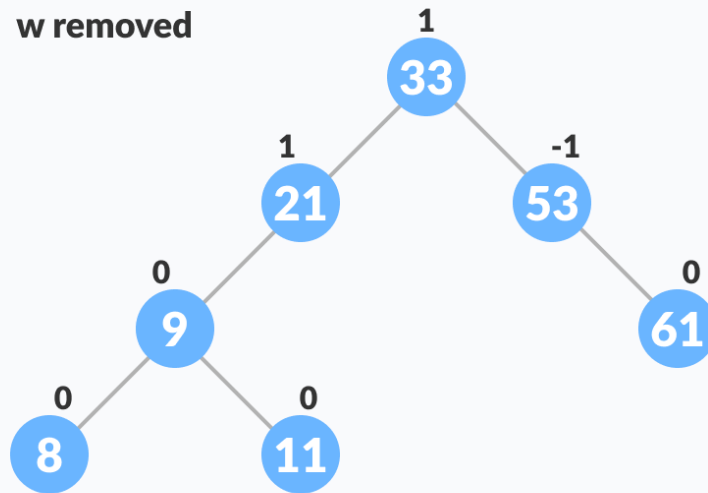
**Substitute the content
of `nodeToBeDeleted`
with that of `w`**



Substitute the node to be

deleted

w removed

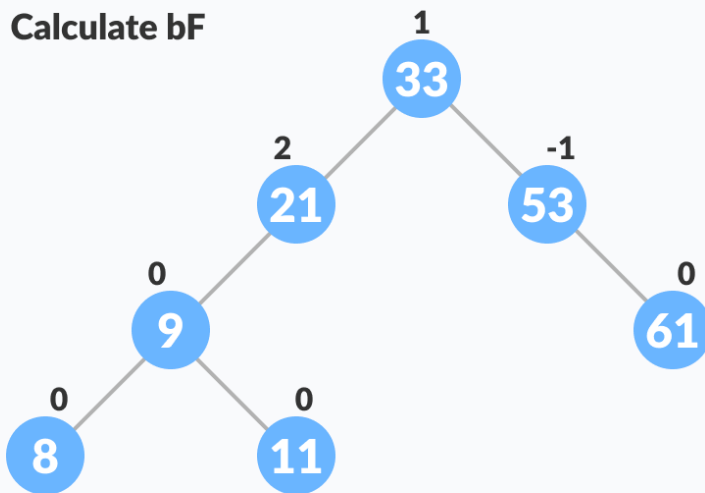


b. Remove the leaf node `w`.

Remove w

3. Update `balanceFactor` of the nodes.

Calculate bF

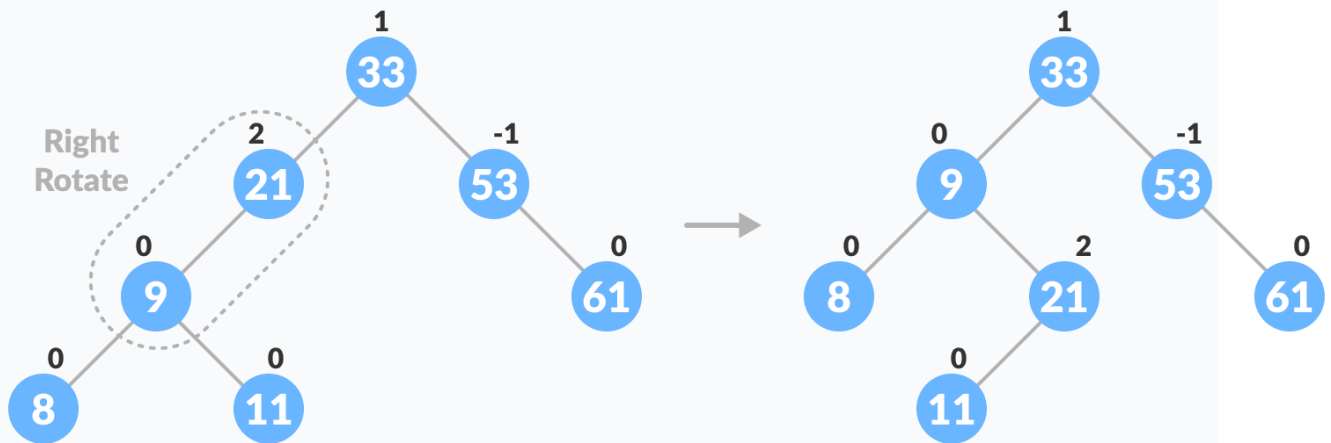


Update bf

4. Rebalance the tree if the balance factor of any of the nodes is not equal to -1, 0 or 1.

a. If `balanceFactor` Of `currentNode` > 1,

a. If `balanceFactor` of `leftChild` ≥ 0 , do right rotation.



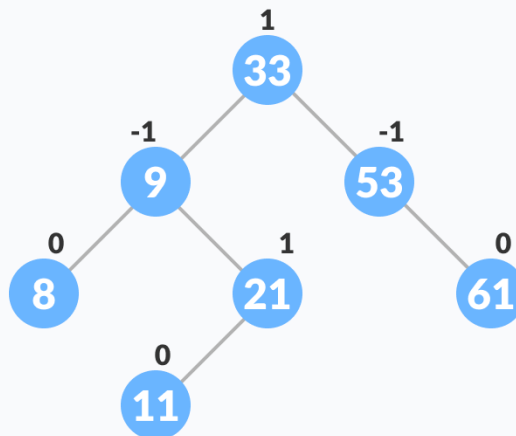
Right-rotate for balancing the tree

b. Else do left-right rotation.

b. If `balanceFactor` of `currentNode` < -1 ,

a. If `balanceFactor` of `rightChild` ≤ 0 , do left rotation.

b. Else do right-left rotation.



5. The final tree is:

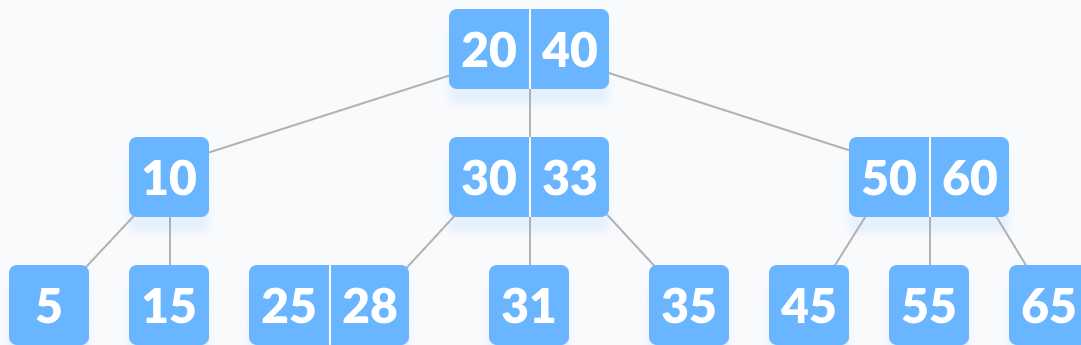
Avl tree final

Construct AVL tree for the following data
21,26,30,9,4,14,28,18,15,10,2,3,7

B-tree

B-tree is a special type of self-balancing search tree in which each node can contain more than one key and can have more than two children. It is a generalized form of the binary search tree.

It is also known as a height-balanced m-way tree.



tree

B-

Why do you need a B-tree data structure?

What is B Tree in Data Structure?

The B Tree is a particular m-way tree which can be used to access discs in a variety of ways. At most m children and m-1 keys can be found in a B-Tree of order m. One of the main advantages of the B tree is its capacity to store a large number of keys inside a single node and huge key values while keeping the tree's height low.

The need for B-tree arose with the rise in the need for lesser time in accessing physical storage media like a hard disk. The secondary storage devices are slower with a larger capacity. There was a need for such types of data structures that minimize the disk access.

Other data structures such as a binary search tree, avl tree, red-black tree, etc can store only one key in one node. If you have to store a large number of keys, then the height of such trees becomes very large, and the access time increases.

However, B-tree can store many keys in a single node and can have multiple child nodes. This decreases the height significantly allowing faster disk accesses.

B Tree Applications

- databases and file systems
- to store blocks of data (secondary storage media)
- multilevel indexing

Importance of B Tree:

B Tree is important because they are a fundamental tool for storing and retrieving large amount of data efficiently.

1. Databases:

B Tree are used to create indexes that help find individual entries in large data sets. Major DBMSs like MySQL, Postgres, and MongoDB use B Tree for efficient data lookups.

2. File Systems :

B Trees are used to manage directories by organizing files so that they can be accessed and modified.

3. Operating System:

B Tree are used to manage memory efficiently.

4. DNS:

B Trees are used to store and retrieve information about domain names.

5. Compilers:

B Trees are used to store symbol tables for efficient compilation of code.